Experiment-19

VETCARE — ANIMAL HEALTHCARE PORTAL

1. Title

VetCare: A Web-Based Animal Healthcare, Veterinary Consultation and Disease Reporting System.

The proposed system connects farmers with government veterinary hospitals (CVHs) and private veterinary doctors. Farmers can register, submit animal complaints with text and supporting media, follow complaint status, receive doctor responses, view disease information, provide feedback and record payment information.

2. Aim

To design and develop a responsive web-based veterinary support portal that enables farmers to communicate animal health signs and symptoms to registered veterinary doctors through text, images, audio and video; allows doctors to review complaints and provide guidance; maintains complaint, user and payment records; and supports area-wise analysis of frequently reported animal diseases.

Provide separate registration and login for farmers and doctors.

Allow farmers to submit complaints and track their status.

Enable doctors to view complaints and post solutions or advice.

Support free access to government CVHs and paid consultations with private clinics.

Maintain disease information, news, feedback and payment records.

Use complaint location and disease labels to prepare area-wise reports.

3. Software Requirements

Hardware requirements

Computer or laptop / smartphone with a modern web browser.

Internet connection for access to a deployed system.

Server and database storage for production deployment.

Camera or microphone when a farmer wants to capture images, audio or video.

Software requirements

Operating system: Windows, Linux, macOS or Android/iOS browser.

Frontend: HTML5, CSS3 and JavaScript (ES6).

Editor: Visual Studio Code or any text editor.

Browser: Google Chrome, Microsoft Edge, Firefox or Safari.

Production backend (recommended): Node.js / Java / PHP with REST APIs.

Production database (recommended): MySQL or PostgreSQL.

Media storage: secure object storage with access control.

Payment integration: an authorized bank/payment gateway and EasyPaisa merchant integration, subject to provider availability and approval.

Prototype note: The code supplied in this assignment is a front-end demonstration. It uses browser localStorage and sessionStorage. It does not provide secure server authentication, a shared database, real file uploads, real bank transfers or live EasyPaisa transactions.

4. Problem Description

Farmers may have difficulty obtaining timely veterinary advice, particularly when a veterinary hospital or qualified doctor is far from their village. Symptoms may be difficult to explain over a telephone call, and farmers may not know which doctor or service is appropriate. Complaint records may also be scattered across paper registers or individual devices, making it difficult to monitor cases and identify disease patterns in a particular area.

VetCare addresses these issues through a central web portal. A farmer creates an account and submits an animal complaint with the animal type, age, location, symptoms and optional media evidence. The doctor receives the complaint, reviews the available details and records a response and status. Farmers can view the response from their dashboard.

Government CVH consultation is represented as a free service option. Private clinic consultations are represented as paid services. Doctors can provide bank transfer or EasyPaisa payment instructions in their profile. The academic prototype only records a payment reference; actual payment processing must be implemented through approved provider APIs and verified server-side.

The system can support area-wise disease analysis when each complaint is consistently tagged with a disease diagnosis and geographic area. A doctor-confirmed diagnosis should be used for reliable statistics; farmer-reported symptoms alone must not be treated as a confirmed disease.

5. Modules

Farmer module

Registration, login, profile, animal complaint submission, media attachment selection, complaint status, doctor details, VetCare information, news, reports, feedback and payment record.

Doctor module

Doctor registration and login, complaint inbox, complaint review, solution/advice entry, status update, cure library management, news management, farmer details, payment details and feedback review.

Complaint management

Generates a unique complaint ID, stores animal details and symptoms, records date and area, lists selected media metadata and tracks status from Submitted to In Review, Advice Provided, Refer to Hospital or Closed.

Media management

Accepts image, audio and video selection in the browser prototype. In production, files must be uploaded to protected storage, scanned, size-limited and linked to a complaint record.

Subscription and payment

Government CVH option is free. Private clinic option requires a fee. Payment instructions may include bank transfer and EasyPaisa details. The prototype records the selected method and transaction reference only.

Disease and cure library

Displays common animal disease information, warning signs and general prevention guidance. Only qualified veterinarians should provide diagnosis and prescriptions.

News and feedback

Allows authorized doctors/admins to publish veterinary news and allows farmers to submit feedback.

Reporting and analytics

Groups doctor-confirmed disease records by area, animal type and date range to identify frequently reported diseases. Reports should show sample size and date range.

Main user functions

Farmer: Register; Login; Post Complaint; My Complaint Status; View VetCare Details; View News; View Doctor; View Report; Post Feedback; Make Payment; My Profile.

Doctor: Register; Login; View Complaint; Post Solution; Manage Cure; Manage News; View User Details; View Payment Details; View Feedback Details.

6. Algorithm

Farmer workflow

Start the application and display the home page.

If the farmer is new, open registration and collect name, email, password, phone and area.

Validate the entered fields and create the farmer account.

Allow the farmer to log in using registered credentials.

Open the farmer dashboard after successful login.

Enter animal type, age, area and signs/symptoms.

Optionally select image, audio or video files as supporting evidence.

Select Government CVH (free) or Private Clinic (paid).

Generate a unique complaint ID and save the complaint record.

Display the complaint ID and current status to the farmer.

When the doctor responds, update the complaint status and display the advice.

If private service is selected, show the doctor's approved payment instructions and verify payment through the payment provider in production.

End.

Doctor workflow

Doctor registers and logs in using an authorized doctor account.

System displays the complaint inbox.

Doctor opens a complaint and reviews animal details, symptoms and attached media.

Doctor enters advice, referral instructions or a request for further information.

Doctor selects the updated complaint status and saves the response.

System stores the response with doctor ID and timestamp.

Farmer views the updated status and doctor response.

End.

Area-wise disease analysis

Read complaint records for the selected date range.

Include only records with a doctor-confirmed diagnosis and a valid area.

Group records by area and diagnosis.

Count cases for each disease in each area.

Sort counts in descending order.

Display the most frequently recorded diseases with the total number of confirmed cases and the date range.

End.

7. Complete HTML, CSS and JavaScript Code

Source code — index.html

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>VetCare - Animal Healthcare Portal</title>

<style>

```
:root{--green:#176b4d;--light:#eaf5ef;--ink:#20332b;--muted:#66756d;--red:#a33a32}
*{box-sizing:border-box} body{margin:0;font-family:Arial,sans-serif;background:#f4f7f5;color:var(--ink)}
header{background:var(--green);color:white;padding:20px 5%;display:flex;justify-content:space-between;align-items:center;gap:12px;flex-wrap:wrap}
header h1{margin:0;font-size:24px} nav{display:flex;gap:8px;flex-wrap:wrap}
button,.btn{border:0;border-radius:7px;padding:10px 14px;background:var(--green);color:white;cursor:pointer;font-weight:bold}
nav button{background:#ffffff20;border:1px solid #ffffff55}
button:hover{opacity:.88}.wrap{max-width:1100px;margin:25px auto;padding:0 16px}
.hero,.panel{background:white;border-radius:12px;padding:22px;margin-bottom:18px;box-shadow:0 2px 12px #173c2b12}
.hero{background:linear-gradient(110deg,#eaf5ef,#fff)} h2{margin-top:0;color:var(--green)}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:14px}
label{display:block;font-weight:bold;margin:12px 0 5px}
input,select,textarea{width:100%;padding:11px;border:1px solid #cbd8d0;border-radius:7px;font:inherit}
textarea{min-height:90px;resize:vertical}.muted{color:var(--muted);font-size:13px}
.hidden{display:none!important}.notice{padding:12px;border-radius:8px;background:var(--light);margin:12px 0}
.danger{background:#fff0ee;color:var(--red)}.item{border:1px solid #dce7e0;padding:14px;border-radius:9px;margin:10px 0}
.tag{display:inline-block;background:var(--light);padding:4px 8px;border-radius:20px;font-size:12px}
footer{text-align:center;color:var(--muted);padding:24px}
table{width:100%;border-collapse:collapse}th,td{padding:10px;border-bottom:1px solid #e1e9e4;text-align:left}
@media(max-width:600px){header{align-items:flex-start}header h1{font-size:20px}}
</style>
</head>
<body>
<header><h1>🐾 VetCare Animal Health Portal</h1><nav>
<button onclick="showPage('home')">Home</button><button onclick="showPage('register')">Register</button>
```

```
<button onclick="showPage('login')">Login</button><button
onclick="showPage('farmer')">Farmer</button>

<button onclick="showPage('doctor')">Doctor</button><button
onclick="showPage('cures')">Cure Library</button>

</nav></header>

<main class="wrap">

<section id="home" class="page">

<div class="hero"><h2>Animal healthcare, closer to every farmer</h2>

<p>Submit animal symptoms using text, photographs, audio or video. A registered doctor can
review the complaint and provide guidance.</p>

<button onclick="showPage('register')">Create an account</button></div>

<div class="grid"><div class="panel"><h3>Submit a complaint</h3><p>Describe
symptoms and attach supporting media.</p></div>

<div class="panel"><h3>Connect with doctors</h3><p>Choose a government veterinary
hospital or a private clinic.</p></div>

<div class="panel"><h3>Learn and prevent</h3><p>Browse basic disease information and
sample care guidance.</p></div></div>

<div class="notice">Demonstration project: records are stored in this browser only. Real
deployment requires a secure server, database, authentication and payment-provider
integration.</div>

</section>

<section id="register" class="page hidden panel"><h2>Farmer / Doctor Registration</h2>

<form id="registerForm"><div class="grid">

<div><label>Full name</label><input id="regName" required></div>

<div><label>Email</label><input id="regEmail" type="email" required></div>

<div><label>Password</label><input id="regPassword" type="password" minlength="6"
required></div>

<div><label>Account type</label><select
id="regRole"><option>Farmer</option><option>Doctor</option></select></div>

<div><label>Phone</label><input id="regPhone" required></div>

<div><label>Village / Area</label><input id="regArea" required></div>

</div><button type="submit">Register</button></form><p id="registerMsg"
class="notice"></p></section>

<section id="login" class="page hidden panel"><h2>Login</h2>
```

<form id="loginForm"><label>Email</label><input id="loginEmail" type="email" required>

<label>Password</label><input id="loginPassword" type="password" required><button>Login</button></form>

</section>

<section id="farmer" class="page hidden">

<h2>Farmer Dashboard</h2> Register or login to use the demo dashboard.

<h3>Post animal complaint</h3><form id="complaintForm">

<label>Animal</label><input id="animal" placeholder="Cow, goat, dog..." required>

<label>Animal age</label><input id="animalAge" placeholder="e.g. 3 years">

<label>Village / Area</label><input id="complaintArea" required>

<label>Preferred service</label><select id="service"><option>Government CVH - Free</option><option>Private clinic - Paid</option></select>

<label>Signs and symptoms</label><textarea id="symptoms" required placeholder="Describe symptoms, duration and changes in behaviour"></textarea>

<label>Upload image, audio or video</label><input id="media" type="file" accept="image/*,audio/*,video/*" multiple>

For this front-end demo, only file names and types are saved. A production server must securely upload and store the actual files.

<button type="submit">Submit complaint</button></form>

<h3>My complaint status</h3> No complaints yet.

<h3>Make payment (demo)</h3><form id="paymentForm"><label>Complaint ID</label><input id="paymentComplaint" required>

<label>Amount (INR)</label><input id="amount" type="number" min="1" required>

<label>Payment method</label><select id="payMethod"><option>Bank transfer</option><option>EasyPaisa</option></select>

<label>Transaction reference</label><input id="txnRef" required><button>Record demo payment</button></form>

<h3>Post feedback</h3><form id="feedbackForm"><label>Feedback</label><textarea id="feedbackText" required></textarea><button>Submit feedback</button></form>

</section>

<section id="doctor" class="page hidden">

<h2>Doctor Dashboard</h2> Demo view of submitted complaints. A real system must enforce doctor authorization on the server.

No complaints submitted.

<h3>Post solution / update status</h3><form id="solutionForm"><label>Complaint ID</label><input id="solutionId" required>

<label>Doctor's advice</label><textarea id="solutionText" required></textarea>

<label>Status</label><select id="solutionStatus"><option>In Review</option><option>Advice Provided</option><option>Refer to Hospital</option><option>Closed</option></select>

<button>Save response</button></form>

<h3>Doctor profile and payment details</h3> Production profile fields should include qualification, registration number, clinic type, bank account details and EasyPaisa account details. Sensitive payment details must be encrypted and access-controlled.

</section>

<section id="cures" class="page hidden panel"><h2>Basic disease and cure library</h2>

Foot-and-mouth disease (FMD) Possible signs: fever, mouth blisters, excessive salivation and lameness. Isolate the animal and contact a veterinarian promptly. Prevention includes vaccination under veterinary guidance. Illustrative information

Mastitis in dairy cattle Possible signs: swollen or painful udder, abnormal milk and reduced yield. A veterinarian should examine the animal and prescribe treatment. Maintain clean milking practices. Illustrative information

Parasitic infestation Possible signs: weight loss, poor coat condition, diarrhoea or anaemia. Diagnosis and deworming schedule should be decided by a veterinarian. Illustrative information

Do not use these examples as prescriptions. Never administer medicines without advice from a qualified veterinarian. </section>

</main><footer>VetCare • Academic front-end prototype</footer>

<script>

const KEY='vetcare_demo_v1';

function readData(){return JSON.parse(localStorage.getItem(KEY)||'{"users":[],"complaints":[],"payments":[],"feedback":[]}')}

```
function saveData(d){localStorage.setItem(KEY,JSON.stringify(d))}

function
showPage(id){document.querySelectorAll('.page').forEach(p=>p.classList.add('hidden'));doc
ument.getElementById(id).classList.remove('hidden');if(id==='farmer')renderFarmer();if(id=
=='doctor')renderDoctor()}

function currentUser(){return JSON.parse(sessionStorage.getItem('vetcare_user')||'null')}

document.getElementById('registerForm').addEventListener('submit',e=>{e.preventDefault();
let
d=readData(),email=regEmail.value.trim().toLowerCase();if(d.users.some(u=>u.email===em
ail)){registerMsg.textContent='This email is already registered.';return}

d.users.push({id:'U'+Date.now(),name:regName.value,email,password:regPassword.value,rol
e:regRole.value,phone:regPhone.value,area:regArea.value});saveData(d);registerMsg.textCo
ntent='Registration successful. Please log in.';e.target.reset()});

document.getElementById('loginForm').addEventListener('submit',e=>{e.preventDefault();le
t
d=readData(),u=d.users.find(x=>x.email===loginEmail.value.trim().toLowerCase()&&x.pas
sword===loginPassword.value);if(!u){loginMsg.textContent='Invalid email or
password.';return}sessionStorage.setItem('vetcare_user',JSON.stringify({id:u.id,name:u.name
,role:u.role}));loginMsg.textContent='Welcome, '+u.name+'. Open the relevant
dashboard.';});

document.getElementById('complaintForm').addEventListener('submit',e=>{e.preventDefaul
t();let u=currentUser();if(!u||u.role!=='Farmer'){complaintMsg.textContent='Please log in
with a Farmer account first.';return}let
d=readData(),files=[...media.files].map(f=>({name:f.name,type:f.type,size:f.size}));let
c={id:'C'+Date.now(),userId:u.id,farmer:u.name,animal:animal.value,age:animalAge.value,ar
ea:complaintArea.value,service:service.value,symptoms:symptoms.value,files,status:'Submitt
ed',doctorReply:'',created:new
Date().toLocaleString()};d.complaints.push(c);saveData(d);complaintMsg.textContent='Com
plaint submitted. Your ID is '+c.id;e.target.reset();renderFarmer()});

function renderFarmer(){let
u=currentUser();if(!u)return;farmerWelcome.textContent='Signed in as '+u.name+'
('+u.role+').';let
d=readData(),list=d.complaints.filter(c=>c.userId===u.id);myComplaints.innerHTML=list.le
ngth?list.map(c=>`<div class="item"><b>${c.id} —
${c.animal}</b><p>${c.symptoms}</p><span
class="tag">${c.status}</span><p>${c.doctorReply||'Awaiting doctor
response'}</p></div>`).join(''):'No complaints yet.'}

function renderDoctor(){let
d=readData();allComplaints.innerHTML=d.complaints.length?d.complaints.map(c=>`<div
class="item"><b>${c.id} — ${c.animal}</b><p>Farmer: ${c.farmer} | Area:
${c.area}</p><p>${c.symptoms}</p><p>Media: ${(c.files||[]).map(f=>f.name).join(',
```

')||'None'}</p>${c.status}<p>${c.doctorReply||'No response yet'}</p> `).join(''):'No complaints submitted.'}

document.getElementById('solutionForm').addEventListener('submit',e=>{e.preventDefault();let d=readData(),c=d.complaints.find(x=>x.id===solutionId.value.trim());if(!c){solutionMsg.textContent='Complaint ID not found.';return}c.doctorReply=solutionText.value;c.status=solutionStatus.value;saveData(d);solutionMsg.textContent='Response saved for '+c.id;renderDoctor()});

document.getElementById('paymentForm').addEventListener('submit',e=>{e.preventDefault();let d=readData();d.payments.push({id:'P'+Date.now(),complaintId:paymentComplaint.value,amount:Number(amount.value),method:payMethod.value,reference:txnRef.value,status:'Demo record only'});saveData(d);paymentMsg.textContent='Demo payment record saved. No money was transferred.';e.target.reset()});

document.getElementById('feedbackForm').addEventListener('submit',e=>{e.preventDefault();let d=readData(),u=currentUser();d.feedback.push({user:u?u.name:'Guest',text:feedbackText.value,date:new Date().toISOString()});saveData(d);feedbackMsg.textContent='Thank you for your feedback.';e.target.reset()});

</script>

</body></html>

8. Sample Input

Farmer registration

Field	Sample value
Name	Ravi Kumar
Email	ravi@example.com
Password	farmer123
Role	Farmer
Phone	9876543210
Area	Anantapur

Animal complaint

Field	Sample value
Animal	Cow
Animal age	4 years
Area	Anantapur

Service	Government CVH - Free
Symptoms	Reduced appetite, fever and mouth sores for two days
Media	cow_symptoms.jpg (example filename)

Doctor response

Complaint ID: C[generated automatically]. Advice: Please isolate the animal and arrange an examination by a veterinarian as soon as possible. Status: Refer to Hospital.

This is example input only. It is not a diagnosis or prescription.

9. Expected Output with Image

After successful registration and login, the farmer dashboard displays the complaint form and complaint-status section. After submission, the application displays a generated complaint ID and status. The doctor dashboard displays the complaint details and allows the doctor to enter a response.

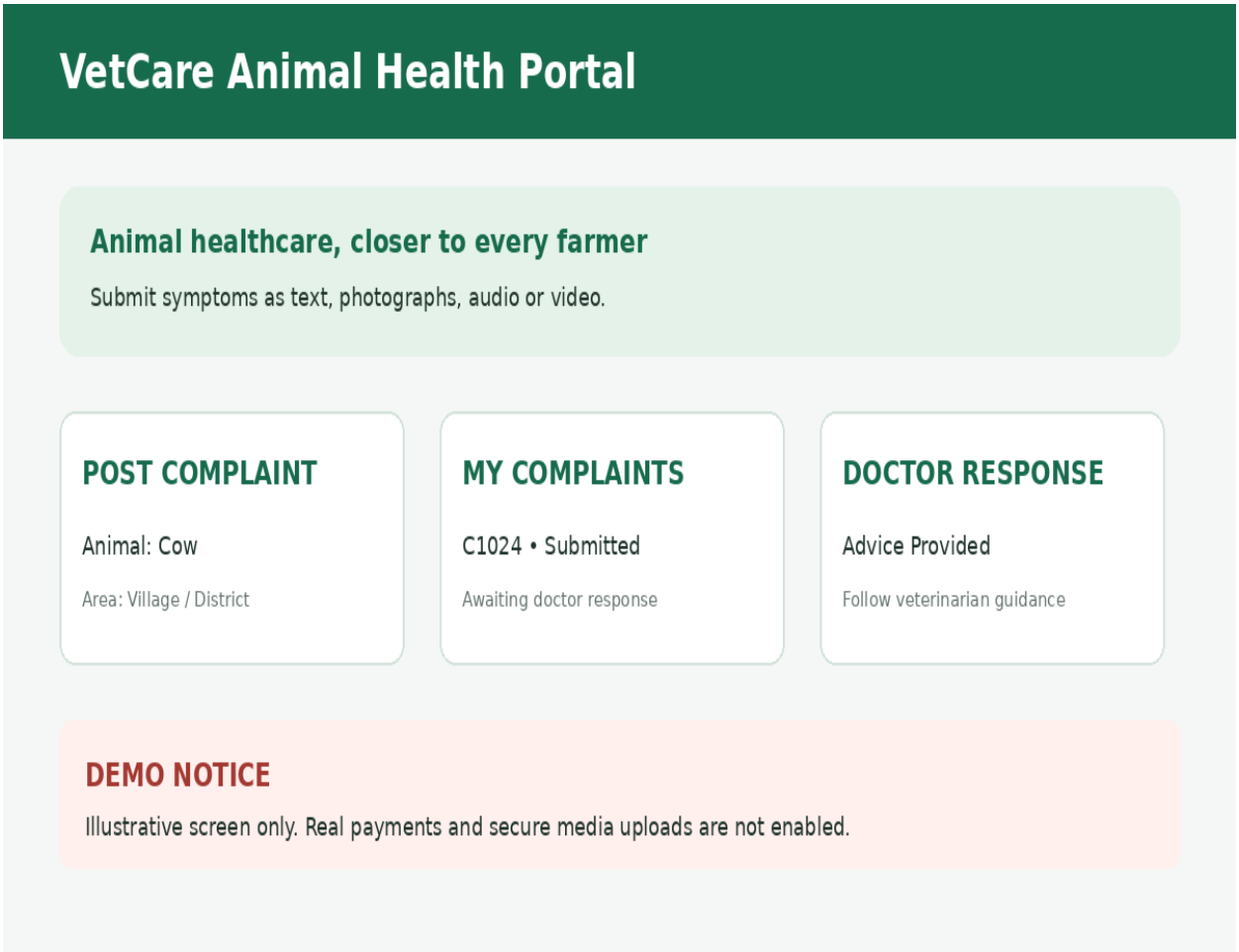


Figure 1. Illustrative VetCare dashboard and expected screen layout (prototype mock-up).

10. Result

The VetCare front-end prototype demonstrates the main workflow of a web-based animal healthcare portal. It provides farmer and doctor registration forms, login, complaint submission, media-file selection, complaint status display, doctor response entry, a basic disease information library, feedback submission and a simulated payment record. The design also describes how a production system can support free government CVH consultations, paid private clinic consultations and area-wise disease reporting.

The prototype is suitable for demonstrating the user interface and basic browser-side flow. Before real-world use, it must be connected to a secure backend and database; passwords must be hashed; user roles and doctor credentials must be verified; uploaded media must be securely stored; and payments must be processed and verified through authorized payment gateways. Veterinary advice and prescriptions must be provided only by qualified professionals.

Experiment-20

RENTRIDE — VEHICLE RENTAL PLATFORM

1. Title

RentRide is an online platform that allows customers to search and book cars, SUVs and trucks. Administrators can add vehicles, update rental details and approve or reject customer booking requests.

2. Aim

To design and develop a responsive vehicle rental website using HTML, CSS and JavaScript that provides customer registration and login, vehicle search with filters, booking requests, booking status tracking, check-in submission, rental history, reviews and administrator management.

Provide a searchable fleet of cars, SUVs and trucks.

Allow customers to select vehicles and submit booking details.

Enable customers to view booking status and rental history.

Allow administrators to add, update and remove vehicle details.

Allow administrators to approve or reject booking requests.

Provide check-in forms, scheduling information and customer reviews.

3. Software Requirements

Hardware requirements

Desktop computer, laptop, tablet or smartphone.

Internet connection for a deployed website.

At least 4 GB RAM and a modern web browser for development and testing.

Software requirements

Operating system: Windows, Linux or macOS.

Frontend: HTML5, CSS3 and JavaScript (ES6).

Code editor: Visual Studio Code or any text editor.

Browser: Google Chrome, Microsoft Edge, Firefox or Safari.

Production backend (recommended): Node.js, Java, PHP or Python with REST APIs.

Production database (recommended): MySQL or PostgreSQL.

Production hosting: Web server with HTTPS and secure database access.

Prototype note: The supplied code is a front-end lab demonstration. It stores data in browser localStorage/sessionStorage. It does not provide real-time fleet availability, server-side authentication, secure payment processing or legally binding booking confirmation.

4. Problem Description

Customers often need to contact multiple rental agencies to compare vehicle types, prices and pickup locations. Manual booking can involve repeated phone calls, paper records and uncertainty about whether a vehicle is available or a booking has been approved. Rental providers also need a simple way to maintain vehicle details, review customer requests and track completed rentals.

RentRide provides a centralized website where customers can register, log in and search a vehicle fleet using filters such as vehicle type, location and maximum daily price. Each vehicle has a description, capacity and rental price. Customers select a vehicle and submit their name, customer number, address, pickup location and rental dates. The system creates a booking ID and initially marks the request as Pending approval.

An administrator logs in to add or manage vehicle records and approve, reject or mark bookings as completed. Customers can view their booking status, submit check-in notes and write reviews. A production version should additionally validate vehicle availability for the complete rental period, prevent overlapping confirmed bookings, integrate secure payments and send real-time email/SMS confirmations.

5. Modules

Search module

Displays available vehicle records and filters by vehicle type, pickup location and maximum daily price. Rental dates are collected for the booking request.

Customer registration and login

Creates a customer account using name, email, password, customer number and address. The demonstration checks credentials in browser storage; production must use secure server-side authentication.

Vehicle details module

Shows vehicle name, type, description, capacity, pickup location and price per day. Administrators can add or delete vehicle entries in the prototype.

Appointment / booking module

Allows a logged-in customer to select a vehicle, enter customer details and rental dates, calculate an estimated rental total and submit a booking request.

Booking management module

Allows the administrator to view all booking requests and change their status to Approved, Rejected or Completed.

Schedule and check-in module

Stores rental dates and accepts customer check-in notes or vehicle-condition details against a booking ID.

Past appointment / rental history

Displays bookings marked as Completed in the customer's dashboard.

Reviews module

Collects a booking ID, star rating and written customer review.

Admin profile and management

Provides a demo administrator login and a dashboard for managing vehicle details and booking requests.

Main user functions

Admin: Login; Add Vehicle; Manage Vehicle Details (type, price, capacity, location and description); Manage Booking Details; Approve User Booking; View Profile.

User: Register; Login; Search Vehicle Details; Select Vehicle and Make Booking (name, customer number, address, vehicle type, location and dates); View Booking Status; My Profile; Submit Check-in Form; View Rental History; Submit Reviews.

6. Algorithm

Customer workflow

Start the RentRide website.

If the customer is new, open the registration page and enter name, email, password, customer number and address.

Validate the fields and save the customer account.

Log in using the registered email and password.

Open the vehicle search page.

Select vehicle type, pickup location and maximum daily price; enter rental dates when ready to book.

Display matching vehicle details and daily rental prices.

Select a vehicle and open the booking form.

Enter customer name, customer number, address, pickup location, start date and end date.

Validate the dates and calculate the estimated rental amount.

Generate a unique booking ID and save the request with Pending approval status.

Display the booking ID and status in the customer dashboard.

After administrator action, show Approved, Rejected or Completed status.

Allow the customer to submit check-in notes and a review.

End.

Administrator workflow

Start the administrator page.

Enter administrator credentials.

Validate the credentials and open the admin dashboard.

Add a vehicle with name, type, price, location, capacity and description, or remove an existing vehicle.

View customer booking requests.

Review the vehicle, customer details and rental dates.

Approve or reject the booking request.

After the rental is completed, update the booking status to Completed.

Save changes and display the updated booking list.

End.

Rental amount calculation

Number of rental days = difference between end date and start date + 1. Estimated total = number of rental days × vehicle price per day. The prototype uses this simple inclusive-day calculation. A real rental business must define its own pickup/return time, partial-day, tax, deposit and late-return rules.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

```
<title>RentRide - Vehicle Rental Platform</title>

<style>

:root{--navy:#172b4d;--blue:#2463a6;--light:#eef4fb;--ink:#243247;--muted:#68778b;--gold:#f0b84b}

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:#f4f7fb;color:var(--ink)}

header{background:var(--navy);color:white;padding:18px 5%;display:flex;justify-content:space-between;align-items:center;gap:12px;flex-wrap:wrap}

header h1{margin:0;font-size:24px}nav{display:flex;gap:8px;flex-wrap:wrap}

button{border:0;border-radius:7px;padding:10px 14px;background:var(--blue);color:white;cursor:pointer;font-weight:bold}

nav button{background:#ffffff20;border:1px solid #ffffff55}button:hover{opacity:.88}

.wrap{max-width:1120px;margin:25px auto;padding:0 16px}.page.hidden{display:none!important}

.hero,.panel,.vehicle{background:white;border-radius:12px;padding:21px;margin-bottom:18px;box-shadow:0 2px 12px #172b4d12}

.hero{background:linear-gradient(110deg,#e8f1fc,#fff)}h2{color:var(--navy);margin-top:0}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:14px}

label{display:block;font-weight:bold;margin:12px 0 5px}

input,select,textarea{width:100%;padding:11px;border:1px solid #cbd6e3;border-radius:7px;font:inherit}

textarea{min-height:80px}.muted{color:var(--muted);font-size:13px}.notice{background:var(--light);padding:12px;border-radius:8px;margin:12px 0}

.item{border:1px solid #dce5ef;border-radius:9px;padding:14px;margin:10px 0}

.tag{display:inline-block;background:#e8f1fc;padding:4px 8px;border-radius:20px;font-size:12px}

.price{color:var(--blue);font-weight:bold}.vehicle-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,1fr));gap:14px}

.vehicle h3{margin:7px 0}.vehicle .emoji{font-size:40px}.danger{background:#fff0ed;color:#9a342c}

footer{text-align:center;color:var(--muted);padding:24px}table{width:100%;border-collapse:collapse}

td,th{padding:10px;border-bottom:1px solid #e2e9f0;text-align:left}

@media(max-width:600px){header h1{font-size:20px}}

</style>
```

```
</head>

<body>

<header><h1>🚗 RentRide</h1><nav>

<button onclick="showPage('home')">Home</button><button
onclick="showPage('register')">Register</button>

<button onclick="showPage('login')">Login</button><button
onclick="showPage('search')">Search Vehicles</button>

<button onclick="showPage('user')">My Dashboard</button><button
onclick="showPage('admin')">Admin</button>

</nav></header>

<main class="wrap">

<section id="home" class="page">

<div class="hero"><h2>Find your ride. Book with confidence.</h2>

<p>Explore cars, SUVs and trucks. Search by vehicle type, rental dates and pickup location,
then submit a booking request.</p>

<button onclick="showPage('search')">Browse vehicles</button></div>

<div class="grid"><div class="panel"><h3>Wide vehicle choice</h3><p>Choose a vehicle
that matches your journey and budget.</p></div>

<div class="panel"><h3>Simple booking</h3><p>Send a booking request and check its
approval status.</p></div>

<div class="panel"><h3>Rental history</h3><p>Review your previous and current booking
records.</p></div></div>

<p class="notice">Academic prototype: data is stored in this browser only. No real vehicle
availability, payment, identity verification or live confirmation is provided.</p>

</section>

<section id="register" class="page hidden panel"><h2>Create Customer Account</h2>

<form id="registerForm"><div class="grid">

<div><label>Full name</label><input id="regName" required></div>

<div><label>Email</label><input id="regEmail" type="email" required></div>

<div><label>Password</label><input id="regPassword" type="password" minlength="6"
required></div>

<div><label>Customer number</label><input id="regPhone" required></div>

<div><label>Address</label><input id="regAddress" required></div></div>
```

<button>Register</button></form> </section>

<section id="login" class="page hidden panel"><h2>Login</h2>

<form id="loginForm"><label>Email</label><input id="loginEmail" type="email" required>

<label>Password</label><input id="loginPassword" type="password" required><button>Login</button></form> </section>

<section id="search" class="page hidden">

<h2>Search Vehicle Details</h2>

<label>Vehicle type</label><select id="filterType"><option value="All">All types</option><option>Car</option><option>SUV</option><option>Truck</option></select>

<label>Pickup location</label><input id="filterLocation" placeholder="Enter city or area">

<label>Rental start date</label><input id="filterStart" type="date">

<label>Rental end date</label><input id="filterEnd" type="date">

<label>Maximum price per day (₹)</label><input id="filterPrice" type="number" min="0" placeholder="Any price">

<button onclick="renderVehicles()">Search</button> <button onclick="clearFilters()">Clear filters</button>

</section>

<section id="user" class="page hidden">

<h2>Customer Dashboard</h2> Please register and log in to manage bookings.

<h3>My bookings / booking status</h3> No bookings yet.

<h3>Past appointment / rental history</h3> No completed rentals yet.

<h3>My profile</h3> Log in to view your profile.

<h3>Submit check-in details</h3><form id="checkinForm">

<label>Booking ID</label><input id="checkinBooking" required>

<label>Check-in notes / vehicle condition</label><textarea id="checkinNotes" required></textarea>

<button>Submit check-in form</button></form>

<h3>Reviews</h3><form id="reviewForm"><label>Booking ID</label><input id="reviewBooking" required>

<label>Rating</label><select id="rating"><option>5</option><option>4</option><option>3</option><option>2</option><option>1</option></select>

<label>Review</label><textarea id="reviewText" required></textarea><button>Submit review</button></form>

</section>

<section id="booking" class="page hidden panel"><h2>Book Selected Vehicle</h2>

<form id="bookingForm"><input id="bookVehicleId" type="hidden">

<label>Customer name</label><input id="bookName" required>

<label>Customer number</label><input id="bookPhone" required>

<label>Address</label><input id="bookAddress" required>

<label>Vehicle type</label><input id="bookType" readonly>

<label>Pickup location</label><input id="bookLocation" required>

<label>Start date</label><input id="bookStart" type="date" required>

<label>End date</label><input id="bookEnd" type="date" required>

<button>Submit booking request</button></form> </section>

<section id="admin" class="page hidden">

<h2>Administrator Login</h2> Demo credentials: admin@rentride.test / admin123

<form id="adminLogin"><label>Email</label><input id="adminEmail" type="email" required>

<label>Password</label><input id="adminPassword" type="password" required><button>Admin login</button></form>

<h3>Add / Manage Vehicle</h3><form id="vehicleForm">

<label>Vehicle name</label><input id="vName" required>

<label>Type</label><select id="vType"><option>Car</option><option>SUV</option><option>Truck</option></select>

<label>Price per day (₹)</label><input id="vPrice" type="number" min="1" required>

<label>Location</label><input id="vLocation" required>

<label>Seats / capacity</label><input id="vCapacity" required>

<label>Description</label><textarea id="vDescription" required></textarea><button>Add vehicle</button></form>

<h3>Manage Booking Details</h3> No booking requests.

</section>

</main><footer>RentRide • Internet Programming Lab Experiment</footer>

<script>

const KEY='rentride_demo_v1';

const initialVehicles=[

{id:'V101',name:'City Hatchback',type:'Car',price:1800,location:'Anantapur',capacity:'5 seats',description:'Compact and economical city car.'},

{id:'V102',name:'Family SUV',type:'SUV',price:3200,location:'Anantapur',capacity:'7 seats',description:'Comfortable SUV for family trips.'},

{id:'V103',name:'Utility Pickup',type:'Truck',price:2800,location:'Kurnool',capacity:'1.5 tonne',description:'Practical pickup for goods transport.'}

];

function readData(){let d=JSON.parse(localStorage.getItem(KEY)||'null');if(!d){d={users:[],vehicles:initialVehicles,bookings:[],checkins:[],reviews:[]};saveData(d)}return d}

function saveData(d){localStorage.setItem(KEY,JSON.stringify(d))}

function currentUser(){return JSON.parse(sessionStorage.getItem('rentride_user')||'null')}

function showPage(id){document.querySelectorAll('.page').forEach(x=>x.classList.add('hidden'));document.getElementById(id).classList.remove('hidden');if(id==='search')renderVehicles();if(id==='user')renderUser();if(id==='admin'&&sessionStorage.getItem('rentride_admin'))showAdminArea()}

function escapeHTML(s){return String(s).replace(/[&<>"']/g,c=>({'&':'&','<':'<','>':'>','"':'"',"'":'''}[c]))}

document.getElementById('registerForm').addEventListener('submit',e=>{e.preventDefault();let

d=readData(),email=regEmail.value.trim().toLowerCase();if(d.users.some(u=>u.email===email)){registerMsg.textContent='Email already registered.';return}

d.users.push({id:'U'+Date.now(),name:regName.value,email,password:regPassword.value,phone:regPhone.value,address:regAddress.value});saveData(d);registerMsg.textContent='Registration successful. Please log in.';e.target.reset()});

document.getElementById('loginForm').addEventListener('submit',e=>{e.preventDefault();let
u=readData().users.find(x=>x.email===loginEmail.value.trim().toLowerCase()&&x.password===loginPassword.value);if(!u){loginMsg.textContent='Invalid email or password.';return}sessionStorage.setItem('rentride_user',JSON.stringify({id:u.id,name:u.name}));loginMsg.textContent='Login successful. Welcome '+u.name+'.'});

function renderVehicles(){let
d=readData(),type=filterType.value,loc=filterLocation.value.trim().toLowerCase(),max=Number(filterPrice.value)||Infinity;

let
list=d.vehicles.filter(v=>(type==='All'||v.type===type)&&(!loc||v.location.toLowerCase().includes(loc))&&v.price<=max);

vehicleResults.innerHTML=list.length?list.map(v=>`<article class="vehicle"><div
class="emoji">${v.type==='Car'?'🚗':v.type==='SUV'?'🚙':'🛻'}</div><h3>${escapeHTML(v.name)}</h3>${escapeHTML(v.type)}<p>${escapeHTML(v.description)} Location: ${escapeHTML(v.location)} | ${escapeHTML(v.capacity)} <p
class="price">₹${v.price} / day</p><button onclick="startBooking('${v.id}')">Select &
Book</button></article>`).join(''):' No vehicles match your filters. '}

function
clearFilters(){filterType.value='All';filterLocation.value='';filterPrice.value='';filterStart.value='';filterEnd.value='';renderVehicles()}

function startBooking(id){let u=currentUser();if(!u){alert('Please log in before
booking.');showPage('login');return}let
v=readData().vehicles.find(x=>x.id===id);bookVehicleId.value=v.id;selectedVehicle.textContent=v.name+' — '+v.type+' —
₹'+v.price+'/day';bookType.value=v.type;bookName.value=u.name;let
user=readData().users.find(x=>x.id===u.id);bookPhone.value=user.phone;bookAddress.value=user.address;bookLocation.value=v.location;showPage('booking')}

document.getElementById('bookingForm').addEventListener('submit',e=>{e.preventDefault();let u=currentUser();if(!u){bookingMsg.textContent='Please log in first.';return}let s=new
Date(bookStart.value),en=new
Date(bookEnd.value);if(!bookStart.value||!bookEnd.value||en<s){bookingMsg.textContent='Please enter valid rental dates.';return}

let
d=readData(),v=d.vehicles.find(x=>x.id===bookVehicleId.value),days=Math.max(1,Math.ce

```
il((en-
s)/86400000)+1),b={id:'B'+Date.now(),userId:u.id,customer:bookName.value,phone:bookPh
one.value,address:bookAddress.value,vehicleId:v.id,vehicle:v.name,type:v.type,location:book
Location.value,start:bookStart.value,end:bookEnd.value,days,total:days*v.price,status:'Pendi
ng approval',created:new
Date().toLocaleString()};d.bookings.push(b);saveData(d);bookingMsg.textContent='Booking
request submitted. Booking ID: '+b.id;e.target.reset()});

function renderUser(){let u=currentUser();if(!u)return;let
d=readData(),mine=d.bookings.filter(b=>b.userId===u.id);welcome.textContent='Signed in
as '+u.name;profileInfo.textContent='Name: '+u.name+' | Email:
'+(d.users.find(x=>x.id===u.id)||{}).email;

myBookings.innerHTML=mine.length?mine.filter(b=>b.status!=='Completed').map(b=>boo
kingCard(b)).join(''):'No active bookings.';

history.innerHTML=mine.filter(b=>b.status==='Completed').map(b=>bookingCard(b)).join('
')||'No completed rentals yet.'}

function bookingCard(b){return `<div class="item"><b>${b.id} —
${escapeHTML(b.vehicle)}</b><p>${b.start} to ${b.end} | ${b.days} day(s) |
₹${b.total}</p><span class="tag">${escapeHTML(b.status)}</span></div>`}

document.getElementById('checkinForm').addEventListener('submit',e=>{e.preventDefault()
;let
d=readData();d.checkins.push({bookingId:checkinBooking.value,notes:checkinNotes.value,d
ate:new Date().toISOString()});saveData(d);checkinMsg.textContent='Check-in form
recorded in this browser demo.';e.target.reset()});

document.getElementById('reviewForm').addEventListener('submit',e=>{e.preventDefault();l
et
d=readData(),u=currentUser();d.reviews.push({bookingId:reviewBooking.value,user:u?u.na
me:'Guest',rating:rating.value,text:reviewText.value});saveData(d);reviewMsg.textContent='
Thank you. Your review has been recorded.';e.target.reset()});

document.getElementById('adminLogin').addEventListener('submit',e=>{e.preventDefault();i
f(adminEmail.value==='admin@rentride.test'&&adminPassword.value==='admin123'){sessi
onStorage.setItem('rentride_admin','yes');adminMsg.textContent='Admin login
successful.';showAdminArea()}else adminMsg.textContent='Invalid demo administrator
credentials.'});

function showAdminArea(){adminArea.classList.remove('hidden');renderAdmin()}

document.getElementById('vehicleForm').addEventListener('submit',e=>{e.preventDefault();
let
d=readData();d.vehicles.push({id:'V'+Date.now(),name:vName.value,type:vType.value,price:
Number(vPrice.value),location:vLocation.value,capacity:vCapacity.value,description:vDescri
ption.value});saveData(d);e.target.reset();renderAdmin();adminMsg.textContent='Vehicle
added successfully.'});
```

```
function renderAdmin(){let
d=readData();adminVehicles.innerHTML=d.vehicles.map(v=>`<div
class="item">${escapeHTML(v.name)} — ${v.type} — ₹${v.price}/day <button
onclick="deleteVehicle('${v.id}')">Delete</button></div>`).join('');

allBookings.innerHTML=d.bookings.length?d.bookings.map(b=>`<div
class="item"><b>${b.id} —
${escapeHTML(b.vehicle)}</b><p>${escapeHTML(b.customer)} | ${b.start} to ${b.end} |
₹${b.total}</p><span class="tag">${b.status}</span><p><button
onclick="setStatus('${b.id}','Approved')">Approve</button> <button
onclick="setStatus('${b.id}','Rejected')">Reject</button> <button
onclick="setStatus('${b.id}','Completed')">Mark
completed</button></p></div>`).join(''):'No booking requests.'}

function deleteVehicle(id){let
d=readData();d.vehicles=d.vehicles.filter(v=>v.id!==id);saveData(d);renderAdmin()}

function setStatus(id,status){let
d=readData(),b=d.bookings.find(x=>x.id===id);if(b)b.status=status;saveData(d);renderAdmi
n()}

renderVehicles();

</script>

</body></html>
```

8. Sample Input

Customer registration

Field	Sample value
Name	Arun Kumar
Email	arun@example.com
Password	arun1234
Customer number	9876543210
Address	Anantapur, Andhra Pradesh

Vehicle search

Filter	Sample value
Vehicle type	SUV
Pickup location	Anantapur
Maximum price per day	₹4,000
Rental dates	10-11-2026 to 12-11-2026

Booking details

Customer name: Arun Kumar | Customer number: 9876543210 | Address: Anantapur, Andhra Pradesh | Vehicle: Family SUV | Type: SUV | Pickup location: Anantapur | Start date: 10-11-2026 | End date: 12-11-2026.

Using the prototype's inclusive-day calculation, this sample is 3 rental days. At ₹3,200 per day, the estimated total is ₹9,600. Sample values are for demonstration only.

9. Expected Output with Image

The home page displays the RentRide platform introduction. The search page shows matching vehicles with their type, location, capacity and price per day. After the customer submits a booking request, a unique booking ID and Pending approval status are displayed. The administrator dashboard lists the request and provides approval, rejection and completion actions.

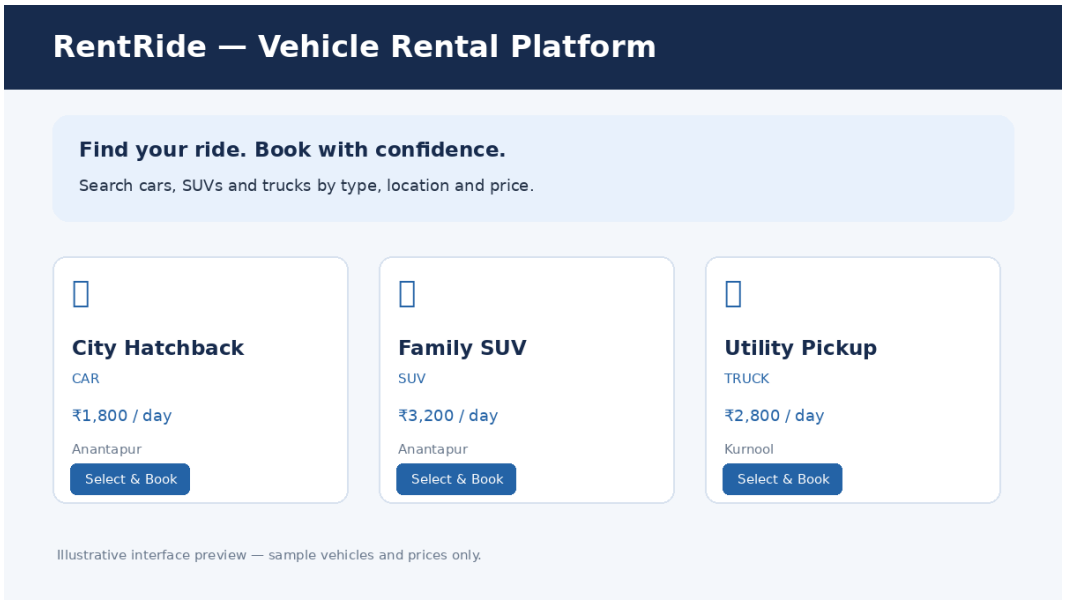


Figure 1. Illustrative RentRide vehicle search and listing page.

10. Result

The RentRide vehicle rental website prototype was designed using HTML, CSS and JavaScript. It demonstrates customer registration and login, vehicle search with filters, vehicle details and pricing, vehicle selection, booking submission, booking ID generation, booking status display, administrator vehicle management, booking approval or rejection, check-in form submission, rental history and customer reviews.

The project meets the core objectives of the Internet Programming lab experiment by demonstrating a customer-facing vehicle rental interface and an administrator management interface. For real-world deployment, the application must be connected to a secure backend and database, implement server-side authentication and role-based access control, validate actual vehicle availability, protect customer data, and integrate an approved payment gateway.

Experiment-21

Online Job Portal

1. TITLE

Design and Development of an Online Job Portal using HTML, CSS and JavaScript.

2. AIM

To design and develop a web-based Online Job Portal that allows applicants to register, search and filter job vacancies, apply online by submitting resume details, and track their applications. The system also allows directors to register, obtain administrator approval, post vacancies and view applicants. The administrator can approve directors, view registered users and generate portal reports.

3. SOFTWARE REQUIREMENTS

Requirement	Specification
Operating System	Windows 10/11, Linux or macOS
Frontend	HTML5, CSS3 and JavaScript
Editor	Visual Studio Code / Notepad++
Browser	Google Chrome, Mozilla Firefox or Microsoft Edge
Storage for demonstration	Browser Local Storage
Suggested production backend	Java / PHP / Node.js with MySQL or another database
Hardware	Computer with 4 GB RAM or above and internet/browser access

4. PROBLEM DESCRIPTION

Job seekers often need to visit several websites or contact companies individually to learn about vacancies. This takes time and effort, and it can be difficult to compare jobs by location and category or to track submitted applications. Employers also need a convenient way to publish vacancies and review candidate resumes.

The proposed Online Job Portal provides a single interface for vacancy discovery and recruitment. Applicants can register, log in, search by job title, location and category, save their resume details, set profile privacy and apply online. Directors can register and, after administrator approval, publish vacancy information such as job title, image, description, location and contact details. Directors can view the users who applied for their vacancies. Administrators can approve directors, view users and review portal reports.

The prototype demonstrates the workflow using browser Local Storage. It is intended for academic demonstration; a real deployment requires server-side authentication, database storage, secure resume uploads, authorization and notification services.

5. MODULES

Admin Module

Admin login; approve or reject director registrations; view registered users; view summary reports showing users, directors, vacancies and applications.

Director Module

Director registration; pending approval status; login after approval; post vacancy with title, category, location, description, contact and optional image URL; view posted vacancies and applicants; logout.

Applicant/User Module

User registration and login; maintain resume reference and profile privacy; search vacancies; filter by location and category; apply with contact details, resume reference and cover message; view application status; logout.

Vacancy Search Module

Display all available vacancies on one interface. Search by title/company and filter by location and job category.

Application Module

Validate application fields, connect the application to the selected vacancy and signed-in applicant, and store a submitted status.

Privacy and Communication Module

Allow a user to choose public or private resume visibility. The prototype stores this preference; real notifications and company-specific privacy permissions must be implemented by the backend.

6. ALGORITHM

Start the application and display the home page.

Load vacancy, user, director and application records from browser Local Storage; display sample vacancies if no records exist.

Applicant: register with name, email, password and resume reference. Save profile privacy preference.

Applicant: log in using registered credentials. Display the user dashboard.

Applicant: search vacancies using a keyword and filter by location and category.

Applicant: select a vacancy, enter contact details, resume reference and cover message, then submit the application.

System: validate required fields, store the application with the selected vacancy ID and set its status to Submitted.

Director: register with company and login details. Keep the account pending until administrator approval.

Admin: log in, review director registrations and approve eligible directors. View users and portal report.

Approved director: log in, post vacancy details and view applicants for vacancies posted by that director.

Save updated records, allow the user to log out, and stop.

7. COMPLETE HTML / CSS / JAVASCRIPT CODE

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>CareerConnect - Online Job Portal</title>

<style>

*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:#f3f6fb;color:#17233b}

header{background:#142b4a;color:white;padding:18px 7%;display:flex;justify-content:space-between;align-items:center;flex-wrap:wrap}

header h1{margin:0;font-size:24px}nav button,.btn{border:0;border-radius:7px;padding:10px 15px;background:#2463a6;color:white;cursor:pointer;margin:3px}

nav button{background:#29496f}main{max-width:1100px;margin:25px auto;padding:0 16px}

.hero{background:#dcecff;padding:28px;border-radius:14px;margin-bottom:20px}

.hero h2{margin-top:0;color:#14385e}.panel,.job{background:white;padding:18px;border-radius:12px;box-shadow:0 3px 12px #142b4a12;margin:14px 0}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,1fr));gap:14px}

input,select,textarea{width:100%;padding:10px;margin:6px 0 12px;border:1px solid #cbd5e1;border-radius:7px}

label{font-weight:bold;font-size:14px}small,.muted{color:#64748b}.hidden{display:none}.job h3{margin:4px 0;color:#174a7e}

.tag{display:inline-block;background:#e6f2ff;color:#17518a;padding:4px 8px;border-radius:20px;font-size:12px;margin:3px}
```

```
button:hover,.btn:hover{opacity:.88}.danger{background:#b83b45}.success{color:#16834a;font-weight:bold}

#message{padding:10px;color:#12633c}

table{width:100%;border-collapse:collapse;background:white}td,th{padding:10px;border-bottom:1px solid #e2e8f0;text-align:left}

footer{text-align:center;padding:24px;color:#64748b}

</style></head>

<body>

<header><h1>CareerConnect</h1><nav>

<button onclick="show('home')">Home</button><button onclick="show('jobs')">Find Jobs</button>

<button onclick="show('userLogin')">User Login</button><button onclick="show('directorLogin')">Director</button>

<button onclick="show('adminLogin')">Admin</button></nav></header>

<main>

<section id="home">

<div class="hero"><h2>Find the right opportunity for your future</h2><p>Search vacancies, apply online and track your applications from one place.</p><button class="btn" onclick="show('jobs')">Explore Vacancies</button></div>

<div class="grid"><div class="panel"><h3>Search & Filter</h3><p>Find jobs by location and category.</p></div><div class="panel"><h3>Easy Applications</h3><p>Submit your resume through a simple form.</p></div><div class="panel"><h3>Privacy Controls</h3><p>Choose whether your profile is visible to recruiters.</p></div></div>

</section>

<section id="jobs" class="hidden"><div class="panel"><h2>Available Vacancies</h2>

<div class="grid"><div><label>Search title/company</label><input id="q" placeholder="e.g. Java Developer" oninput="renderJobs()"></div>

<div><label>Location</label><select id="loc" onchange="renderJobs()"><option value="">All locations</option><option>Chennai</option><option>Bengaluru</option><option>Hyderabad</option></select></div>

<div><label>Category</label><select id="cat" onchange="renderJobs()"><option value="">All categories</option><option>IT</option><option>Marketing</option><option>Finance</option><option>Healthcare</option></select></div></div>

<div id="jobList"></div></div></section>
```

```
<section id="userRegister" class="hidden"><div class="panel"><h2>User
Registration</h2><label>Full name</label><input
id="regName"><label>Email</label><input id="regEmail"
type="email"><label>Password</label><input id="regPass"
type="password"><label>Resume filename / link</label><input id="resume"
placeholder="resume.pdf"><label>Profile privacy</label><select id="privacy"><option
value="public">Visible to approved directors</option><option
value="private">Private</option></select><button class="btn"
onclick="registerUser()">Create Account</button></div></section>

<section id="userLogin" class="hidden"><div class="panel"><h2>User
Login</h2><label>Email</label><input id="loginEmail"><label>Password</label><input
id="loginPass" type="password"><button class="btn"
onclick="userLogin()">Login</button><button
onclick="show('userRegister')">Register</button></div></section>

<section id="userDash" class="hidden"><div class="panel"><h2>User Dashboard</h2><p
id="welcome"></p><label>Resume</label><input id="dashResume"><label>Profile
privacy</label><select id="dashPrivacy"><option value="public">Visible to approved
directors</option><option value="private">Private</option></select><button class="btn"
onclick="saveProfile()">Save Profile</button><h3>My Applications</h3><div
id="myApps"></div><button onclick="logout()">Logout</button></div></section>

<section id="apply" class="hidden"><div class="panel"><h2>Apply for Vacancy</h2><p
id="applyJob"></p><label>Full name</label><input id="appName"><label>Contact
number</label><input id="appPhone"><label>Email</label><input
id="appEmail"><label>Resume filename / link</label><input
id="appResume"><label>Cover message</label><textarea id="cover"></textarea><button
class="btn" onclick="submitApplication()">Submit Application</button></div></section>

<section id="directorRegister" class="hidden"><div class="panel"><h2>Director
Registration</h2><label>Name</label><input
id="dirName"><label>Company</label><input
id="dirCompany"><label>Email</label><input
id="dirEmail"><label>Password</label><input id="dirPass" type="password"><button
class="btn" onclick="registerDirector()">Register for Approval</button></div></section>

<section id="directorLogin" class="hidden"><div class="panel"><h2>Director
Login</h2><label>Email</label><input
id="dirLoginEmail"><label>Password</label><input id="dirLoginPass"
type="password"><button class="btn" onclick="directorLogin()">Login</button><button
onclick="show('directorRegister')">Register</button></div></section>

<section id="directorDash" class="hidden"><div class="panel"><h2>Director
Dashboard</h2><p id="dirWelcome"></p><h3>Post Vacancy</h3><label>Job
title</label><input id="jobTitle"><label>Category</label><select
id="jobCat"><option>IT</option><option>Marketing</option><option>Finance</option><
option>Healthcare</option></select><label>Location</label><input
id="jobLoc"><label>Salary / description</label><textarea
```

```
id="jobDesc"></textarea><label>Contact email</label><input
id="jobContact"><label>Image URL (optional)</label><input id="jobImage"><button
class="btn" onclick="postJob()">Post Vacancy</button><h3>My Vacancies and
Applicants</h3><div id="directorJobs"></div><button
onclick="logout()">Logout</button></div></section>

<section id="adminLogin" class="hidden"><div class="panel"><h2>Admin
Login</h2><label>Email</label><input id="adminEmail"><label>Password</label><input
id="adminPass" type="password"><button class="btn"
onclick="adminLogin()">Login</button><p class="muted">Demo: admin@career.test /
admin123</p></div></section>

<section id="adminDash" class="hidden"><div class="panel"><h2>Admin
Dashboard</h2><h3>Director Approval</h3><div
id="directorApprovals"></div><h3>Registered Users</h3><div
id="users"></div><h3>Portal Report</h3><p id="report"></p><button
onclick="logout()">Logout</button></div></section>

<p id="message"></p></main><footer>CareerConnect · Online Job Portal
Demonstration</footer>

<script>

const seed=[

{id:1,title:'Junior Java
Developer',company:'TechNova',category:'IT',location:'Chennai',description:'Entry-level
Java, SQL and web development role. Full-time.',contact:'hr@technova.example'},

{id:2,title:'Digital Marketing Executive',company:'Bright
Media',category:'Marketing',location:'Bengaluru',description:'Plan campaigns, manage social
media and analyze results.',contact:'jobs@bright.example'},

{id:3,title:'Accounts
Assistant',company:'FinEdge',category:'Finance',location:'Hyderabad',description:'Support
bookkeeping, invoices and monthly reports.',contact:'careers@finedge.example'}

];

let jobs=JSON.parse(localStorage.jobs||'null')||seed,
users=JSON.parse(localStorage.users||'[]'), directors=JSON.parse(localStorage.directors||'[]'),
apps=JSON.parse(localStorage.apps||'[]');

let currentUser=null,currentDirector=null,currentJob=null;

const
save=()=>{localStorage.jobs=JSON.stringify(jobs);localStorage.users=JSON.stringify(users)
;localStorage.directors=JSON.stringify(directors);localStorage.apps=JSON.stringify(apps)};

function show(id){document.querySelectorAll('main
section').forEach(s=>s.classList.add('hidden'));document.getElementById(id).classList.remov
e('hidden');document.getElementById('message').textContent='';if(id==='jobs')renderJobs();if
```

```
(id==='adminDash')renderAdmin();if(id==='directorDash')renderDirector();if(id==='userDash')renderUser()}

function msg(t){document.getElementById('message').textContent=t}

function renderJobs(){let q=(document.getElementById('q')?.value||'').toLowerCase(),l=document.getElementById('loc')?.value||'',c=document.getElementById('cat')?.value||'';let list=jobs.filter(j=>(j.title+j.company).toLowerCase().includes(q)&&(!l||j.location===l)&&(!c||j.category===c));document.getElementById('jobList').innerHTML=list.length?list.map(j=>`<article class="job"><h3>${j.title}</h3><b>${j.company}</b><p>${j.description}</p><span class="tag">${j.location}</span><span class="tag">${j.category}</span><p><small>Contact: ${j.contact}</small></p><button class="btn" onclick="startApply(${j.id})">Apply Now</button></article>`).join(''):'<p>No matching vacancies found.</p>'}

function registerUser(){let name=regName.value,email=regEmail.value.toLowerCase(),password=regPass.value;if(!name||!email||!password)return msg('Please complete all required fields.');if(users.some(u=>u.email===email))return msg('Email already registered.');users.push({name,email,password,resume:resume.value,privacy:privacy.value});save();msg('Registration successful. Please log in.');show('userLogin')}

function userLogin(){let u=users.find(x=>x.email===loginEmail.value.toLowerCase()&&x.password===loginPass.value);if(!u)return msg('Invalid user login.');currentUser=u;show('userDash')}

function renderUser(){if(!currentUser)return;welcome.textContent='Welcome, '+currentUser.name;dashResume.value=currentUser.resume||'';dashPrivacy.value=currentUser.privacy||'public';myApps.innerHTML=apps.filter(a=>a.email===currentUser.email).map(a=>`<p>${a.title} — <b>${a.status}</b></p>`).join('')||'No applications submitted yet.'}

function saveProfile(){currentUser.resume=dashResume.value;currentUser.privacy=dashPrivacy.value;save();msg('Profile updated.')}

function startApply(id){if(!currentUser){msg('Please log in or register before applying.');return show('userLogin')}currentJob=jobs.find(j=>j.id===id);applyJob.textContent=currentJob.title+' at '+currentJob.company;appName.value=currentUser.name;appEmail.value=currentUser.email;appResume.value=currentUser.resume||'';show('apply')}

function submitApplication(){if(!appName.value||!appPhone.value||!appEmail.value||!appResume.value)return msg('Complete your contact details and resume.');apps.push({id:Date.now(),jobId:currentJob.id,title:currentJob.title,company:currentJob.company,name:appName.value,phone:appPhone.value,email:appEmail.value,resume:app
```

```
Resume.value,cover:cover.value,status:'Submitted'});save();msg('Application submitted successfully.');show('userDash')}

function registerDirector(){let email=dirEmail.value.toLowerCase();if(!dirName.value||!dirCompany.value||!email||!dirPass.value)return msg('Complete all director details.');directors.push({name:dirName.value,company:dirCompany.value,email,password:dirPass.value,approved:false});save();msg('Registration sent to admin for approval.')}

function directorLogin(){let d=directors.find(x=>x.email===dirLoginEmail.value.toLowerCase()&&x.password===dirLoginPass.value&&x.approved);if(!d)return msg('Login failed. Account may be pending admin approval.');currentDirector=d;show('directorDash')}

function postJob(){if(!jobTitle.value||!jobLoc.value||!jobDesc.value||!jobContact.value)return msg('Complete vacancy details.');jobs.push({id:Date.now(),title:jobTitle.value,company:currentDirector.company,category:jobCat.value,location:jobLoc.value,description:jobDesc.value,contact:jobContact.value,image:jobImage.value,owner:currentDirector.email});save();msg('Vacancy posted.');renderDirector()}

function renderDirector(){if(!currentDirector)return;dirWelcome.textContent='Company: '+currentDirector.company;let own=jobs.filter(j=>j.owner===currentDirector.email);directorJobs.innerHTML=own.map(j=>{let a=apps.filter(x=>x.jobId===j.id);return `<article class="job"><b>${j.title}</b> · ${j.location}<p>${a.length} applicant(s)</p>${a.map(x=>`<p>${x.name} | ${x.email} | ${x.phone} | Resume: ${x.resume}</p>`).join('')||'<small>No applicants yet.</small>'}</article>`}).join('')||'No vacancies posted.'}

function adminLogin(){if(adminEmail.value==='admin@career.test'&&adminPass.value==='admin123')show('adminDash');else msg('Invalid admin credentials.')}

function renderAdmin(){directorApprovals.innerHTML=directors.map((d,i)=>`<p>${d.name} — ${d.company} — ${d.approved?'Approved':'Pending'} ${d.approved?'':`<button class="btn" onclick="approve(${i})">Approve</button>`}</p>`).join('')||'No director registrations.';users.innerHTML=users.map(u=>`<p>${u.name} — ${u.email} — ${u.privacy} profile</p>`).join('')||'No registered users.';report.textContent=`${users.length} users · ${directors.length} directors · ${jobs.length} vacancies · ${apps.length} applications.`}

function approve(i){directors[i].approved=true;save();renderAdmin()}

function logout(){currentUser=null;currentDirector=null;show('home');msg('Logged out.')}

show('home');

</script></body></html>
```

8. SAMPLE INPUT

The following sample values can be used to test the application:

Test	Input	Expected action
Applicant registration	Name: Anu Kumar; Email: anu@example.com; Password: user123; Resume: anu_resume.pdf; Privacy: Public	Applicant account is created.
Vacancy search	Keyword: Java; Location: Chennai; Category: IT	Junior Java Developer vacancy is displayed.
Job application	Name: Anu Kumar; Phone: 9876543210; Email: anu@example.com; Resume: anu_resume.pdf	Application is submitted and appears in My Applications.
Director registration	Name: Ravi; Company: TechNova; Email: director@technova.example; Password: dir123	Director account is marked Pending approval.
Admin login	Email: admin@career.test; Password: admin123	Admin dashboard opens and director can be approved.
Vacancy posting	Title: Java Developer; Category: IT; Location: Chennai; Description: Java and SQL role; Contact: hr@technova.example	Vacancy is added to the portal.

9. EXPECTED OUTPUT WITH IMAGE

The home page displays the CareerConnect portal header, vacancy search entry point and featured vacancy cards. Applicants can filter vacancies and apply. Directors see their posted vacancies and applicants after login. Administrators see director approval controls, registered users and report totals.

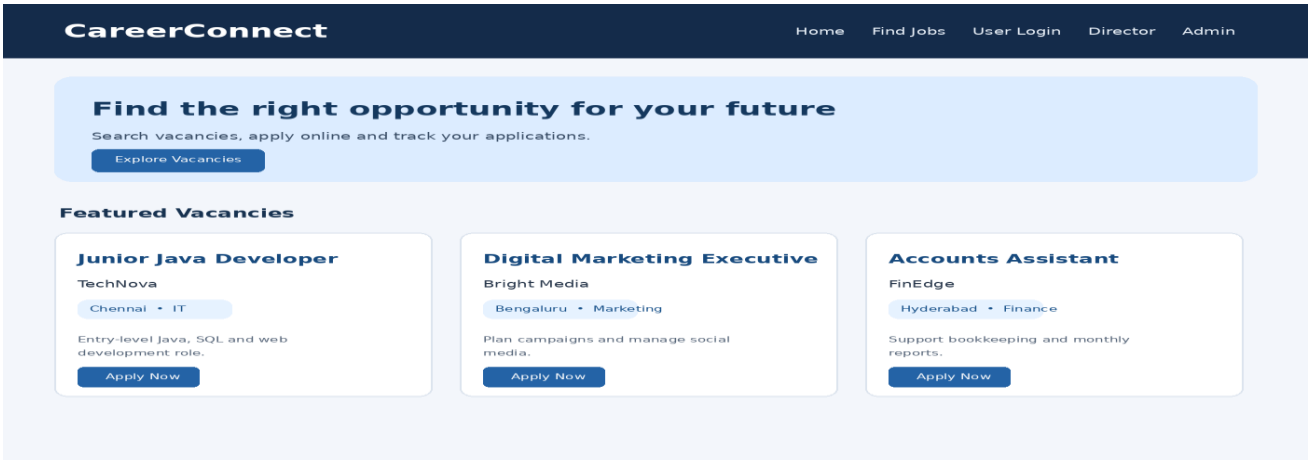


Figure 1: Illustrative home page showing the vacancy portal and featured jobs.

10. RESULT

Thus, the Online Job Portal prototype was designed and developed using HTML, CSS and JavaScript. It demonstrates applicant registration and login, vacancy search and filtering, online job application, director registration and approval, vacancy posting, applicant viewing, administrator user management and basic reporting. The project provides a foundation that can be extended with a secure backend, database, resume file upload, email/SMS notifications, company-specific privacy controls and online deployment.

Experiment-22
SMART SOLID WASTE MANAGEMENT SYSTEM

1. Title

Smart Solid Waste Management System Using Data Analytics and Route Optimization

2. Aim

The aim of this project is to develop a Smart Solid Waste Management System for efficient management of garbage segregation, collection, transportation, public complaints, driver activities, reports, and optimized garbage-truck routes. The system centralizes data in a database and presents useful information through dashboards for administrators, workforce members, and clients.

3. Software Requirements

Operating System: Windows 10/11, Linux, or macOS

Frontend: HTML5, CSS3 and JavaScript

Backend (for full deployment): Node.js, PHP, or Python

Database: MySQL or MongoDB

Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox

Development Tool: Visual Studio Code

Maps Integration: Google Maps API

Optional Server Environment: XAMPP or Node.js

4. Problem Description

Traditional waste-management systems often use fixed collection schedules and manual monitoring. This can cause overflowing garbage bins, delayed collection, unnecessary vehicle movement, inefficient routes, and poor complaint management. Administrators also face difficulties in monitoring drivers and generating accurate reports.

The proposed system digitizes the complete process. Waste management is broadly organized into segregation, collection, and transportation. The server collects operational data and stores it in a centralized database. The data can be analyzed and displayed through dashboards. Reports help administrators monitor collection activities and complaints. Based on garbage-bin locations and collection needs, route optimization algorithms and Google Maps API can be used to assign efficient routes to garbage trucks, ending at the dumping site.

5. Modules

5.1 Administrator Module

Login

Create garbage bins

Update or delete garbage bins

Assign the best route for drivers

Manage driver information

View garbage reports

View complaints submitted by the public

5.2 General Public Module

Register a new account

Login

Register a complaint

View my complaints and complaint status

5.3 Driver Module

Login

Check daily work updates

Choose or view the best route

Update garbage load

6. Algorithm

Nearest Neighbor Route Optimization Algorithm

The prototype uses the Nearest Neighbor algorithm. The garbage truck starts from its current position and repeatedly selects the nearest unvisited garbage bin. After visiting all required bins, the final destination is the dumping site.

1. Start from the truck's current location.

2. Select garbage bins that require collection.

3. Mark all selected bins as unvisited.

4. Calculate the distance from the current location to every unvisited bin.

5. Choose the nearest garbage bin.

6. Add the selected bin to the route and mark it as visited.

7. Set that bin as the current location.

8. Repeat until all required bins are visited.

9. Finally add the dumping site to the route.

Distance Formula: $d = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$

For real-world implementation, Google Maps Directions API can provide road-based navigation, while Dijkstra's Algorithm, A* Algorithm, or Travelling Salesman Problem heuristics can be used for more advanced route optimization.

7. Complete HTML, CSS and JavaScript Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart Solid Waste Management</title>
<style>
* { box-sizing: border-box; font-family: Arial, sans-serif; }
body { margin: 0; background: #f4f7f6; color: #222; }
header { background: #1f6f54; color: white; padding: 18px; text-align: center; }
.container { width: 92%; max-width: 1100px; margin: 25px auto; }
.card {
background: white; padding: 20px; margin: 15px 0;
border-radius: 10px; box-shadow: 0 2px 8px rgba(0,0,0,.12);
}
.grid {
display: grid; grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
gap: 15px;
```

```
}
.stat { background: #e8f5ef; padding: 18px; border-radius: 8px; text-align: center; }
.stat h2 { color: #1f6f54; }
input, textarea {
width: 100%; padding: 10px; margin: 8px 0 15px;
border: 1px solid #bbb; border-radius: 5px;
}
button {
background: #1f6f54; color: white; border: none;
padding: 10px 16px; border-radius: 5px; cursor: pointer; margin: 4px;
}
table { width: 100%; border-collapse: collapse; margin-top: 10px; }
th, td { border: 1px solid #ddd; padding: 10px; }
th { background: #1f6f54; color: white; }
.hidden { display: none; }
.route-box { background: #eef7f3; padding: 15px; margin-top: 10px; }
</style>
</head>
<body>

<header>
<h1>Smart Solid Waste Management System</h1>
<p>Segregation | Collection | Transportation | Analytics</p>
</header>

<div class="container">

<div id="loginCard" class="card">
<h2>Login</h2>
<button onclick="login('admin')">Administrator</button>
<button onclick="login('public')">General Public</button>
<button onclick="login('driver')">Driver</button>
</div>

<div id="dashboard" class="hidden">

<div class="card">
<h2 id="dashboardTitle">Dashboard</h2>
<button onclick="logout()">Logout</button>
</div>

<div class="grid">
<div class="stat"><h3>Total Bins</h3><h2 id="binCount">0</h2></div>
<div class="stat"><h3>Complaints</h3><h2 id="complaintCount">0</h2></div>
<div class="stat"><h3>Garbage Load</h3><h2 id="loadCount">0 kg</h2></div>
<div class="stat"><h3>Route Stops</h3><h2 id="routeCount">0</h2></div>
```

```
</div>

<div id="adminModule" class="card hidden">
<h2>Administrator - Garbage Bin Management</h2>
<input id="binName" placeholder="Garbage Bin Name">
<input id="binX" type="number" placeholder="Location X">
<input id="binY" type="number" placeholder="Location Y">
<button onclick="addBin()">Create Garbage Bin</button>
<button onclick="generateRoute()">Assign Best Route</button>

<table>
<thead>
<tr><th>Bin</th><th>X</th><th>Y</th><th>Action</th></tr>
</thead>
<tbody id="binTable"></tbody>
</table>

<div id="routeOutput" class="route-box">No route generated.</div>
</div>

<div id="publicModule" class="card hidden">
<h2>General Public - Register Complaint</h2>
<input id="complaintName" placeholder="Your Name">
<textarea id="complaintText" placeholder="Enter complaint"></textarea>
<button onclick="addComplaint()">Submit Complaint</button>
<h3>My Complaints & Status</h3>
<ul id="complaintList"></ul>
</div>

<div id="driverModule" class="card hidden">
<h2>Driver - Daily Work Update</h2>
<p>Follow the assigned route and collect waste.</p>
<input id="garbageLoad" type="number" placeholder="Garbage Load in kg">
<button onclick="updateLoad()">Update Garbage Load</button>
<p id="driverMessage"></p>
</div>

<div id="reportModule" class="card hidden">
<h2>Garbage Report</h2>
<p id="reportText"></p>
</div>

</div>
</div>

<script>
```

```
let bins = [
{name: "Bin A", x: 2, y: 3},
{name: "Bin B", x: 8, y: 4},
{name: "Bin C", x: 5, y: 9}
];

let complaints = [];
let totalLoad = 0;

function login(role) {
document.getElementById("loginCard").classList.add("hidden");
document.getElementById("dashboard").classList.remove("hidden");

document.getElementById("dashboardTitle").innerText =
role.toUpperCase() + " DASHBOARD";

document.getElementById("adminModule").classList.toggle("hidden", role !== "admin");
document.getElementById("publicModule").classList.toggle("hidden", role !== "public");
document.getElementById("driverModule").classList.toggle("hidden", role !== "driver");
document.getElementById("reportModule").classList.toggle("hidden", role !== "admin");

renderBins();
updateDashboard();
}

function logout() {
document.getElementById("dashboard").classList.add("hidden");
document.getElementById("loginCard").classList.remove("hidden");
}

function addBin() {
const name = document.getElementById("binName").value;
const x = Number(document.getElementById("binX").value);
const y = Number(document.getElementById("binY").value);

if (!name) {
alert("Enter garbage bin name");
return;
}

bins.push({name, x, y});
renderBins();
updateDashboard();

document.getElementById("binName").value = "";
document.getElementById("binX").value = "";
```

```
document.getElementById("binY").value = "";
}

function deleteBin(index) {
bins.splice(index, 1);
renderBins();
updateDashboard();
}

function renderBins() {
const table = document.getElementById("binTable");
table.innerHTML = "";

bins.forEach((bin, index) => {
table.innerHTML += `
<tr>
<td>${bin.name}</td>
<td>${bin.x}</td>
<td>${bin.y}</td>
<td><button onclick="deleteBin(${index})">Delete</button></td>
</tr>`;
});
}

function distance(a, b) {
return Math.sqrt((a.x-b.x)**2 + (a.y-b.y)**2);
}

// Nearest Neighbor Route Algorithm
function generateRoute() {
if (bins.length === 0) return;

let unvisited = [...bins];
let current = {name: "Truck Start", x: 0, y: 0};
let route = [current.name];

while (unvisited.length > 0) {
let nearestIndex = 0;
let shortest = distance(current, unvisited[0]);

for (let i = 1; i < unvisited.length; i++) {
let d = distance(current, unvisited[i]);
if (d < shortest) {
shortest = d;
nearestIndex = i;
}
```

```
}

current = unvisited.splice(nearestIndex, 1)[0];
route.push(current.name);
}

route.push("Dumping Site");

document.getElementById("routeOutput").innerHTML =
"<b>Optimized Route:</b> " + route.join(" → ");

document.getElementById("routeCount").innerText = route.length;
}

function addComplaint() {
const name = document.getElementById("complaintName").value;
const text = document.getElementById("complaintText").value;

if (!name || !text) {
alert("Complete all fields");
return;
}

complaints.push({name, text, status: "Submitted"});

document.getElementById("complaintName").value = "";
document.getElementById("complaintText").value = "";

renderComplaints();
updateDashboard();
}

function renderComplaints() {
const list = document.getElementById("complaintList");
list.innerHTML = "";

complaints.forEach(c => {
list.innerHTML +=
`<li><b>${c.name}:</b> ${c.text} - Status: ${c.status}</li>`;
});
}

function updateLoad() {
const load = Number(document.getElementById("garbageLoad").value);

if (load <= 0) {
```

```
alert("Enter valid load");
return;
}

totalLoad += load;

document.getElementById("driverMessage").innerText =
"Garbage load updated successfully: " + load + " kg";

document.getElementById("garbageLoad").value = "";
updateDashboard();
}

function updateDashboard() {
document.getElementById("binCount").innerText = bins.length;
document.getElementById("complaintCount").innerText = complaints.length;
document.getElementById("loadCount").innerText = totalLoad + " kg";

document.getElementById("reportText").innerText =
"Total Bins: " + bins.length +
" | Total Complaints: " + complaints.length +
" | Total Garbage Collected: " + totalLoad + " kg";
}
</script>

</body>
</html>
```

8. Sample Input

Garbage Bin A: X = 2, Y = 3

Garbage Bin B: X = 8, Y = 4

Garbage Bin C: X = 5, Y = 9

Public Complaint: Garbage bin near the market is overflowing.

Driver Garbage Load: 250 kg

Truck Starting Point: (0, 0)

Final Destination: Dumping Site

9. Expected Output with Image

The system displays role-based dashboards. The administrator can manage garbage bins and generate an optimized route. The general public can submit complaints and view their status. Drivers can check work updates and update garbage loads.

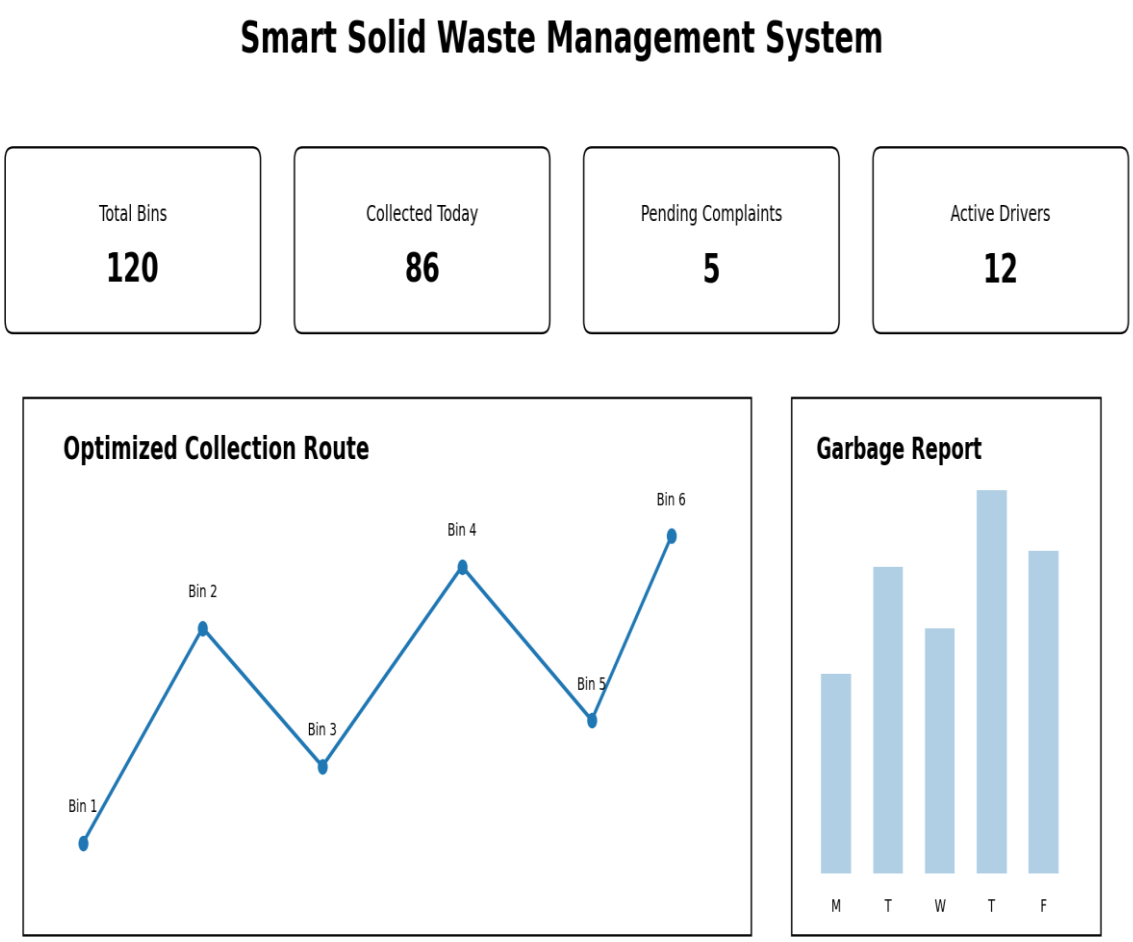


Figure: Expected Smart Solid Waste Management Dashboard

10. Result

The Smart Solid Waste Management System was successfully designed as a web-based prototype using HTML, CSS, and JavaScript. The system supports administrator, general-public, and driver modules. It provides garbage-bin management, complaint registration, garbage-load updates, dashboard statistics, reporting, and route optimization using the Nearest Neighbor algorithm. The system can be enhanced by connecting it to a database, integrating Google Maps API, using GPS tracking, adding IoT smart-bin sensors, and applying advanced data analytics.

Experiment 23:

BLOOD MANAGEMENT INFORMATION SYSTEM

1. Title

Blood Management Information System

2. Aim

The aim of this project is to develop a web-based Blood Management Information System that provides quick and secure access to blood donor and blood-bank records. The system helps donors, hospitals, and administrators manage blood-related information efficiently, monitor blood availability, generate useful reports, and support planning and decision-making for improved medical service delivery.

3. Software Requirements

Operating System: Windows 10/11, Linux, or macOS

Frontend: HTML5, CSS3, and JavaScript

Backend (for full implementation): Node.js / PHP / Python

Database: MySQL or MongoDB

Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox

Development Environment: Visual Studio Code

Web Server: XAMPP, Apache, or Node.js

Optional API: Google Maps API for locating nearby blood banks

4. Problem Description

Blood banks and hospitals require accurate and timely information about blood donors, blood groups, available blood units, and blood-bank facilities. Manual record management can make searching difficult and may delay access to important information during urgent situations.

The proposed Blood Management Information System provides centralized access to donor records collected from different locations. It supports registration and login facilities for donors and hospitals. Donors can post and manage their information, search for donor details, and search blood-bank information. Hospitals can register, maintain blood-bank information, update blood-group availability, and view blood-bank details. The system can also generate reports that help management monitor activities and make informed decisions.

5. Modules

5.1 Donor/User Module

Register a new donor account

Login securely

Post donor information

View my donor information

Search donor details

Search blood-bank details

Logout

5.2 Hospital Module

Register a hospital account

Login securely

Post blood-bank information

Update blood-group availability status

View blood-bank details

Logout

5.3 Administrator/Management Module (Optional Extension)

Monitor donor registrations

Monitor blood-bank information

Generate reports

View blood-group availability

Analyze donation activity

Support planning and decision-making

6. Algorithm

Blood Donor Search and Blood Availability Algorithm

1. Start the application.
2. Accept the user role: Donor/User or Hospital.
3. Authenticate the user using login credentials.
4. Store registration and blood-related information in the database.
5. When a donor search is requested, filter donor records by blood group and location.
6. Display matching donor records.
7. When blood-bank information is requested, search available blood-bank records.
8. Hospitals update the available quantity or status of each blood group.
9. Generate reports from donor registrations and blood availability data.
10. Display the result on the dashboard and end the requested operation.

Search Logic: For each record, compare the requested blood group and/or location with the stored record. Matching records are returned to the user.

7. Complete HTML, CSS and JavaScript Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Blood Management Information System</title>
<style>
* { box-sizing: border-box; font-family: Arial, sans-serif; }

body {
margin: 0;
background: #f5f5f5;
color: #222;
}

header {
background: #b71c1c;
color: white;
padding: 20px;
text-align: center;
}

.container {
width: 92%;
max-width: 1100px;
margin: 25px auto;
}

.card {
background: white;
padding: 20px;
margin: 15px 0;
border-radius: 10px;
box-shadow: 0 2px 8px rgba(0,0,0,.12);
}

.grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
gap: 15px;
}
```

```
.stat {
background: #ffebee;
padding: 18px;
text-align: center;
border-radius: 8px;
}

.stat h2 {
color: #b71c1c;
}

input, select {
width: 100%;
padding: 10px;
margin: 8px 0 15px;
border: 1px solid #bbb;
border-radius: 5px;
}

button {
background: #b71c1c;
color: white;
border: none;
padding: 10px 16px;
margin: 4px;
border-radius: 5px;
cursor: pointer;
}

button:hover {
background: #7f0000;
}

table {
width: 100%;
border-collapse: collapse;
margin-top: 12px;
}

th, td {
border: 1px solid #ddd;
padding: 10px;
text-align: left;
}

th {
```

```
background: #b71c1c;
color: white;
}

.hidden {
display: none;
}
</style>
</head>

<body>

<header>
<h1>Blood Management Information System</h1>
<p>Donor Records | Blood Banks | Blood Availability</p>
</header>

<div class="container">

<div id="loginCard" class="card">
<h2>Select User Role</h2>
<button onclick="login('donor')">Donor / User</button>
<button onclick="login('hospital')">Hospital</button>
</div>

<div id="dashboard" class="hidden">

<div class="card">
<h2 id="dashboardTitle">Dashboard</h2>
<button onclick="logout()">Logout</button>
</div>

<div class="grid">
<div class="stat">
<h3>Total Donors</h3>
<h2 id="donorCount">0</h2>
</div>

<div class="stat">
<h3>Blood Banks</h3>
<h2 id="bankCount">0</h2>
</div>

<div class="stat">
<h3>Available Units</h3>
<h2 id="unitCount">0</h2>
```

```
</div>
</div>

<!-- DONOR MODULE -->
<div id="donorModule" class="card hidden">
<h2>Donor / User Information</h2>

<input id="donorName" placeholder="Donor Name">
<input id="donorLocation" placeholder="Location">

<select id="donorBlood">
<option value="">Select Blood Group</option>
<option>A+</option>
<option>A-</option>
<option>B+</option>
<option>B-</option>
<option>AB+</option>
<option>AB-</option>
<option>O+</option>
<option>O-</option>
</select>

<button onclick="addDonor()">Post Donor Information</button>

<h3>Search Donor Details</h3>
<select id="searchBlood">
<option value="">Select Blood Group</option>
<option>A+</option>
<option>A-</option>
<option>B+</option>
<option>B-</option>
<option>AB+</option>
<option>AB-</option>
<option>O+</option>
<option>O-</option>
</select>

<button onclick="searchDonors()">Search Donor</button>

<table>
<thead>
<tr>
<th>Name</th>
<th>Blood Group</th>
<th>Location</th>
</tr>
```

```
</thead>
<tbody id="donorTable"></tbody>
</table>
</div>

<!-- HOSPITAL MODULE -->
<div id="hospitalModule" class="card hidden">
<h2>Hospital / Blood Bank Information</h2>

<input id="bankName" placeholder="Blood Bank Name">
<input id="bankLocation" placeholder="Blood Bank Location">

<select id="bankBlood">
<option value="">Select Blood Group</option>
<option>A+</option>
<option>A-</option>
<option>B+</option>
<option>B-</option>
<option>AB+</option>
<option>AB-</option>
<option>O+</option>
<option>O-</option>
</select>

<input id="bloodUnits" type="number"
placeholder="Available Blood Units">

<button onclick="addBloodBank()">Post / Update Blood Bank Info</button>

<table>
<thead>
<tr>
<th>Blood Bank</th>
<th>Location</th>
<th>Blood Group</th>
<th>Units</th>
</tr>
</thead>
<tbody id="bankTable"></tbody>
</table>
</div>

<!-- SEARCH BLOOD BANK -->
<div id="searchModule" class="card hidden">
<h2>Search Blood Bank Details</h2>
<button onclick="showBanks()">View Blood Bank Details</button>
```

```
<p id="searchResult"></p>
</div>

</div>
</div>

<script>
let donors = [
{ name: "Ravi", blood: "O+", location: "Hyderabad" },
{ name: "Anil", blood: "A+", location: "Kurnool" }
];

let bloodBanks = [
{
name: "City Blood Bank",
location: "Hyderabad",
blood: "O+",
units: 50
}
];

function login(role) {
document.getElementById("loginCard").classList.add("hidden");
document.getElementById("dashboard").classList.remove("hidden");

document.getElementById("dashboardTitle").innerText =
role === "donor" ? "DONOR / USER DASHBOARD" : "HOSPITAL DASHBOARD";

document.getElementById("donorModule")
.classList.toggle("hidden", role !== "donor");

document.getElementById("hospitalModule")
.classList.toggle("hidden", role !== "hospital");

document.getElementById("searchModule")
.classList.remove("hidden");

renderDonors(donors);
renderBanks();
updateDashboard();
}

function logout() {
document.getElementById("dashboard").classList.add("hidden");
document.getElementById("loginCard").classList.remove("hidden");
}
```

```
function addDonor() {
const name = document.getElementById("donorName").value;
const blood = document.getElementById("donorBlood").value;
const location = document.getElementById("donorLocation").value;

if (!name || !blood || !location) {
alert("Please enter all donor details.");
return;
}

donors.push({ name, blood, location });

document.getElementById("donorName").value = "";
document.getElementById("donorBlood").value = "";
document.getElementById("donorLocation").value = "";

renderDonors(donors);
updateDashboard();
}

function renderDonors(list) {
const table = document.getElementById("donorTable");
table.innerHTML = "";

list.forEach(donor => {
table.innerHTML += `
<tr>
<td>${donor.name}</td>
<td>${donor.blood}</td>
<td>${donor.location}</td>
</tr>`;
});
}

function searchDonors() {
const blood = document.getElementById("searchBlood").value;

if (!blood) {
renderDonors(donors);
return;
}

const result = donors.filter(donor => donor.blood === blood);
renderDonors(result);
}
```

```
function addBloodBank() {
const name = document.getElementById("bankName").value;
const location = document.getElementById("bankLocation").value;
const blood = document.getElementById("bankBlood").value;
const units = Number(document.getElementById("bloodUnits").value);

if (!name || !location || !blood || units < 0) {
alert("Please enter valid blood bank details.");
return;
}

bloodBanks.push({ name, location, blood, units });

document.getElementById("bankName").value = "";
document.getElementById("bankLocation").value = "";
document.getElementById("bankBlood").value = "";
document.getElementById("bloodUnits").value = "";

renderBanks();
updateDashboard();
}

function renderBanks() {
const table = document.getElementById("bankTable");
table.innerHTML = "";

bloodBanks.forEach(bank => {
table.innerHTML += `
<tr>
<td>${bank.name}</td>
<td>${bank.location}</td>
<td>${bank.blood}</td>
<td>${bank.units}</td>
</tr>`;
});
}

function showBanks() {
const result = bloodBanks.map(bank =>
bank.name + " - " + bank.location +
" - " + bank.blood +
" - Units: " + bank.units
).join("<br>");

document.getElementById("searchResult").innerHTML =
```

```
result || "No blood bank information available.";
}

function updateDashboard() {
const totalUnits = bloodBanks.reduce(
(total, bank) => total + Number(bank.units), 0
);

document.getElementById("donorCount").innerText = donors.length;
document.getElementById("bankCount").innerText = bloodBanks.length;
document.getElementById("unitCount").innerText = totalUnits;
}
</script>

</body>
</html>
```

8. Sample Input

Donor Name: Ravi Kumar

Blood Group: O+

Location: Hyderabad

Search Blood Group: O+

Blood Bank Name: City Blood Bank

Blood Bank Location: Hyderabad

Available Blood Group: O+

Available Blood Units: 50

9. Expected Output with Image

After selecting a user role, the system displays the corresponding dashboard. Donors can add and search donor information, while hospitals can post blood-bank details and update blood-group availability. Dashboard statistics display the number of registered donors, blood banks, and available blood units.

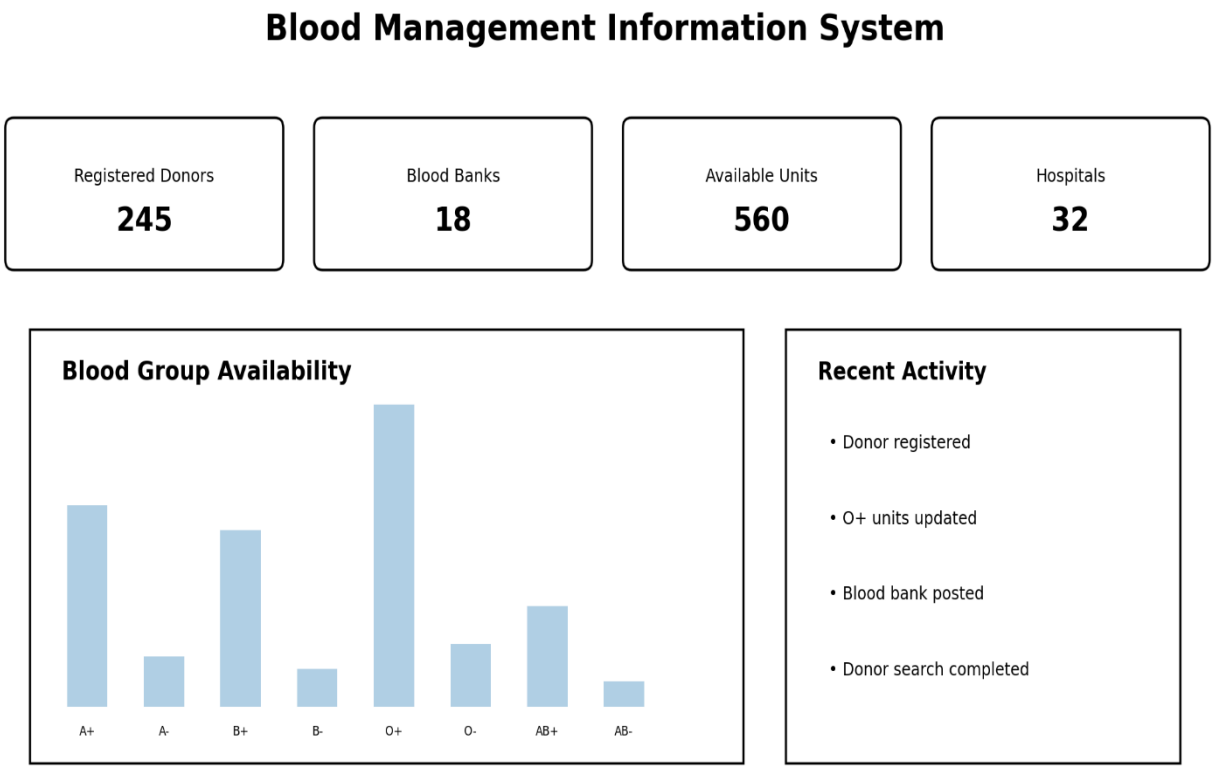


Figure: Expected Output of Blood Management Information System

10. Result

The Blood Management Information System was successfully designed as a web-based prototype using HTML, CSS, and JavaScript. The application supports donor and hospital modules, including registration-related data entry, donor information management, donor searching, blood-bank information management, and blood-group availability updates. The system improves access to blood-related information and can support faster planning and decision-making.

Experiment 24:

1. Title

Design and Development of an Online Travel Agency Website – Wanderlust Travels using HTML, CSS and JavaScript.

2. Aim

To design and develop an interactive online travel agency website that enables users to register, log in, explore tourism destinations, view food, hostel/hotel and route details, and book accommodation. The website also provides administrative functions to maintain tourism information and view customer bookings.

3. Software Requirements

Component	Requirement
Frontend	HTML5
Styling	CSS3
Client-side scripting	JavaScript (ES6)
Code editor	Visual Studio Code / any text editor
Web browser	Google Chrome / Microsoft Edge / Firefox
Data storage	Browser LocalStorage (demonstration only)
Operating system	Windows / Linux / macOS

4. Problem Description

Trip planning often requires travellers to search several websites or contact different service providers to compare destinations, accommodation, food facilities and transport routes. This process can be time-consuming and inconvenient. Wanderlust Travels addresses this problem by presenting essential travel information in one web application.

The system has two roles:

User: registers and logs in, browses tourism, food, accommodation and route information, and submits accommodation bookings.

Admin: logs in to the administration area, adds tourism and route information, manages food and hotel details, and reviews bookings.

This lab implementation is a front-end prototype. It stores demonstration data in the browser's LocalStorage. It does not provide production-grade authentication, server-side access control, payment processing or a real booking service.

5. Modules

5.1 User Modules

User Registration

User Login

View Tourism / Destination Details

View Food Details

View Hostel / Hotel Details

View Route Details

My Book Accommodation

View booking confirmation

5.2 Admin Modules

Admin Login

Post Tourism Details

Manage Food Details

Manage Hotel Details

Manage Route Details

View Booking Details

6. Algorithm

Start the application by opening index.html in a web browser.

Initialize sample destination, food, hotel and route data in LocalStorage if no data exists.

Display the home page with travel options and navigation links.

For a new user, collect name, email and password and save the demo account in LocalStorage.

For login, validate the entered credentials against the locally stored demo account or the demo admin credentials.

After user login, allow the user to view destinations, food, hotels and routes.

When the user submits an accommodation booking, validate the required fields and save the booking record.

When admin logs in, display forms to add tourism, food, hotel and route details and a table of bookings.

Update the displayed information whenever a record is added.

Stop when the user closes the browser or navigates away.

7. Complete HTML, CSS and JavaScript Code

<!DOCTYPE html>

<html lang="en">

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Wanderlust Travels</title>
<style>
:root{--navy:#12304a;--blue:#1677a8;--pale:#eef5fa;--ink:#243746}
*{box-sizing:border-box}
body{margin:0;font-family:Arial,sans-serif;background:#f5f8fb;color:var(--ink)}
header{background:var(--navy);color:white;padding:18px 5%;display:flex;
justify-content:space-between;align-items:center;gap:15px;flex-wrap:wrap}
header h1{margin:0;font-size:25px}
nav{display:flex;gap:10px;flex-wrap:wrap}
nav button{background:transparent;color:white;border:0;cursor:pointer;padding:8px}
nav button:hover{text-decoration:underline}
main{max-width:1100px;margin:25px auto;padding:0 18px}
.hero{background:linear-gradient(120deg,#dcecf7,#c5e2ee);padding:45px 30px;
border-radius:15px;margin-bottom:25px}
.hero h2{font-size:32px;color:var(--navy);margin:0 0 12px}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:16px}
.card,.panel{background:white;border:1px solid #dce5ec;border-radius:12px;
padding:18px;box-shadow:0 3px 12px #12304a0c}
.card h3{margin-top:0;color:var(--navy)}
button,.btn{background:var(--blue);color:white;border:0;border-radius:7px;
padding:10px 14px;cursor:pointer}
button:hover,.btn:hover{background:#105d83}
input,select,textarea{width:100%;padding:10px;margin:6px 0 12px;
border:1px solid #cbd6df;border-radius:6px;font:inherit}
label{font-weight:bold;font-size:14px}
form{max-width:520px}
.panel{margin:15px 0}
```

```
.hidden{display:none!important}
.notice{padding:10px;background:#e4f5e9;border-radius:7px;margin:12px 0}
.muted{color:#617586;font-size:14px}
table{width:100%;border-collapse:collapse;background:white}
th,td{padding:10px;border:1px solid #dce5ec;text-align:left}
th{background:#eaf2f7}
footer{text-align:center;background:var(--navy);color:white;padding:18px;margin-top:35px}
@media(max-width:600px){.hero h2{font-size:25px}header{align-items:flex-start}}
</style>
</head>
<body>
<header>
<h1>✈ Wanderlust Travels</h1>
<nav>
<button onclick="showPage('home')">Home</button>
<button onclick="showPage('destinations')">Destinations</button>
<button onclick="showPage('food')">Food</button>
<button onclick="showPage('hotels')">Hotels</button>
<button onclick="showPage('routes')">Routes</button>
<button onclick="showPage('booking')">Book Stay</button>
<button onclick="showPage('login')">Login</button>
<button onclick="logout()">Logout</button>
</nav>
</header>
<main>
<div id="message" aria-live="polite"></div>
<section id="home" class="page">
<div class="hero">
<h2>Discover places. Create memories.</h2>
<p>Explore destinations, compare stays and plan your next journey with Wanderlust Travels.</p>
```

```
<button onclick="showPage('destinations')">Explore Trips</button>
</div>
<h2>Popular travel options</h2>
<div id="homeCards" class="grid"></div>
<p class="muted">Register or log in to make a sample accommodation booking.</p>
</section>
<section id="register" class="page hidden">
<div class="panel"><h2>User Registration</h2>
<form id="registerForm">
<label>Full name</label><input id="regName" required>
<label>Email</label><input id="regEmail" type="email" required>
<label>Password</label><input id="regPassword" type="password" minlength="4" required>
<button type="submit">Register</button>
</form>
</div>
</section>
<section id="login" class="page hidden">
<div class="panel"><h2>Login</h2>
<form id="loginForm">
<label>Email</label><input id="loginEmail" type="email" required>
<label>Password</label><input id="loginPassword" type="password" required>
<button type="submit">Login</button>
</form>
<p class="muted">New user? <a href="#" onclick="showPage('register')">Register here</a>.</p>
<p class="muted">Demo admin: admin@wanderlust.com / admin123</p>
</div>
</section>
<section id="destinations" class="page hidden">
<h2>Tourism Destinations</h2><div id="destinationList" class="grid"></div>
```

```
</section>
<section id="food" class="page hidden">
<h2>Food Details</h2><div id="foodList" class="grid"></div>
</section>
<section id="hotels" class="page hidden">
<h2>Hostel and Hotel Details</h2><div id="hotelList" class="grid"></div>
</section>
<section id="routes" class="page hidden">
<h2>Route Details</h2><div id="routeList" class="grid"></div>
</section>
<section id="booking" class="page hidden">
<div class="panel"><h2>Book Accommodation</h2>
<form id="bookingForm">
<label>Your name</label><input id="guestName" required>
<label>Email</label><input id="guestEmail" type="email" required>
<label>Destination</label><select id="bookDestination" required></select>
<label>Hotel / Hostel</label><select id="bookHotel" required></select>
<label>Check-in date</label><input id="checkin" type="date" required>
<label>Number of guests</label><input id="guests" type="number" min="1" value="1"
required>
<button type="submit">Confirm Booking</button>
</form>
<div id="bookingConfirmation"></div>
</div>
</section>
<section id="admin" class="page hidden">
<h2>Admin Dashboard</h2>
<div class="grid">
<div class="panel"><h3>Post Tourism Details</h3>
<form id="destinationForm">
<label>Destination</label><input id="destName" required>
```

```
<label>Description</label><textarea id="destDesc" required></textarea>
<button>Add Destination</button>
</form>
</div>
<div class="panel"><h3>Manage Food Details</h3>
<form id="foodForm">
<label>Food / cuisine</label><input id="foodName" required>
<label>Location</label><input id="foodLocation" required>
<button>Add Food</button>
</form>
</div>
<div class="panel"><h3>Manage Hotel Details</h3>
<form id="hotelForm">
<label>Hotel / hostel</label><input id="hotelName" required>
<label>Destination</label><input id="hotelPlace" required>
<label>Price per night (₹)</label><input id="hotelPrice" type="number" min="0" required>
<button>Add Hotel</button>
</form>
</div>
<div class="panel"><h3>Manage Route Details</h3>
<form id="routeForm">
<label>From</label><input id="routeFrom" required>
<label>To</label><input id="routeTo" required>
<label>Transport / duration</label><input id="routeInfo" required>
<button>Add Route</button>
</form>
</div>
</div>
<div class="panel"><h3>View Booking Details</h3><div
id="adminBookings"></div></div>
</section>
```

```
</main>
<footer>© 2026 Wanderlust Travels | Plan • Explore • Enjoy</footer>
<script>
const seed = {
destinations:[
{name:"Ooty",desc:"A scenic hill station known for gardens and cool weather."},
{name:"Goa",desc:"Beaches, coastal food and relaxed holiday experiences."},
{name:"Mysuru",desc:"A heritage city famous for its palace and culture."}
],
food:[
{name:"South Indian Meals",location:"Ooty"},
{name:"Goan Fish Curry",location:"Goa"},
{name:"Mysore Masala Dosa",location:"Mysuru"}
],
hotels:[
{name:"Hill View Residency",place:"Ooty",price:1800},
{name:"Coastal Comfort Stay",place:"Goa",price:2500},
{name:"Heritage Inn",place:"Mysuru",price:1500}
],
routes:[
{from:"Bengaluru",to:"Ooty",info:"Bus / car – approx. 6 hours"},
{from:"Panaji",to:"Calangute",info:"Taxi – approx. 45 minutes"},
{from:"Bengaluru",to:"Mysuru",info:"Train / bus – approx. 3 hours"}
],
users:[], bookings:[]
};
function read(key){return JSON.parse(localStorage.getItem(key)||"null");}
function write(key,value){localStorage.setItem(key,JSON.stringify(value));}
function init(){
Object.keys(seed).forEach(k=>{if(!read(k))write(k,seed[k]);});
```

```
renderAll();
}
function data(k){return read(k)||[];}
function showMessage(t){document.getElementById("message").innerHTML=
'<div class="notice">'+escapeHTML(t)+'</div>'; }
function escapeHTML(s){return String(s).replace(/[&<>"']/g,c=>(
{'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]));}
function showPage(id){
document.querySelectorAll(".page").forEach(p=>p.classList.add("hidden"));
const target=document.getElementById(id);
if(target)target.classList.remove("hidden");
if(id==="admin" && sessionStorage.getItem("role")!=="admin"){
showMessage("Please log in as admin to access the dashboard.");showPage("login");return;
}
if(id==="booking" && sessionStorage.getItem("role")!=="user"){
showMessage("Please log in as a user before booking.");showPage("login");return;
}
renderAll();
}
function cards(items,mapper){
return items.map(x=>'<article class="card">'+mapper(x)+'</article>').join("")||
'<p>No details available yet.</p>';
}
function renderAll(){
const destinations=data("destinations"),foods=data("food"),
hotels=data("hotels"),routes=data("routes");
document.getElementById("homeCards").innerHTML=cards(destinations.slice(0,3),
x=>'<h3>'+escapeHTML(x.name)+'</h3><p>'+escapeHTML(x.desc)+'</p>');
document.getElementById("destinationList").innerHTML=cards(destinations,
x=>'<h3>'+escapeHTML(x.name)+'</h3><p>'+escapeHTML(x.desc)+'</p>');
```

```
document.getElementById("foodList").innerHTML=cards(foods,
x=>'<h3>'+escapeHTML(x.name)+'</h3><p>Location: '+escapeHTML(x.location)+'</p>');
document.getElementById("hotelList").innerHTML=cards(hotels,
x=>'<h3>'+escapeHTML(x.name)+'</h3><p>'+escapeHTML(x.place)+
'</p><p>₹'+Number(x.price).toLocaleString('en-IN')+' per night</p>');
document.getElementById("routeList").innerHTML=cards(routes,
x=>'<h3>'+escapeHTML(x.from)+' → '+escapeHTML(x.to)+'</h3><p>'+escapeHTML(x.info)+'</p>');
document.getElementById("bookDestination").innerHTML=destinations.map(x=>
'<option>'+escapeHTML(x.name)+'</option>').join("");
document.getElementById("bookHotel").innerHTML=hotels.map(x=>
'<option>'+escapeHTML(x.name)+'</option>').join("");
const bookings=data("bookings");
document.getElementById("adminBookings").innerHTML=bookings.length?
'<table><tr><th>Guest</th><th>Destination</th><th>Hotel</th><th>Check-in</th><th>Guests</th></tr>'+
bookings.map(b=>'<tr><td>'+escapeHTML(b.name)+'</td><td>'+escapeHTML(b.destination)+
'</td><td>'+escapeHTML(b.hotel)+'</td><td>'+escapeHTML(b.date)+
'</td><td>'+escapeHTML(b.guests)+'</td></tr>').join("")+'</table>':
'<p>No bookings have been made.</p>';
}
document.getElementById("registerForm").addEventListener("submit",e=>{
e.preventDefault();
const users=data("users"),email=document.getElementById("regEmail").value.trim().toLowerCase();
if(users.some(u=>u.email===email)){showMessage("An account with this email already exists.");return;}
users.push({name:document.getElementById("regName").value.trim(),email,
password:document.getElementById("regPassword").value});
write("users",users);showMessage("Registration successful. Please log in.");showPage("login");
```

```
});
document.getElementById("loginForm").addEventListener("submit",e=>{
e.preventDefault();
const email=document.getElementById("loginEmail").value.trim().toLowerCase();
const password=document.getElementById("loginPassword").value;
if(email==="admin@wanderlust.com"&&password==="admin123"){
sessionStorage.setItem("role","admin");showMessage("Admin login
successful.");showPage("admin");return;
}
const user=data("users").find(u=>u.email===email&&u.password===password);
if(user){sessionStorage.setItem("role","user");sessionStorage.setItem("userName",user.name)
;
showMessage("Welcome, "+user.name+"!");showPage("home");}
else showMessage("Invalid email or password. Please try again.");
});
function logout(){sessionStorage.clear();showMessage("You have been logged
out.");showPage("home");}
function addRecord(formId,key,recordFn){
document.getElementById(formId).addEventListener("submit",e=>{
e.preventDefault();const list=data(key);list.push(recordFn());write(key,list);
e.target.reset();renderAll();showMessage("Record added successfully.");
});
}
addRecord("destinationForm","destinations",()=>({
name:document.getElementById("destName").value.trim(),
desc:document.getElementById("destDesc").value.trim()}));
addRecord("foodForm","food",()=>({
name:document.getElementById("foodName").value.trim(),
location:document.getElementById("foodLocation").value.trim()}));
addRecord("hotelForm","hotels",()=>({
name:document.getElementById("hotelName").value.trim(),
```

```
place:document.getElementById("hotelPlace").value.trim(),
price:Number(document.getElementById("hotelPrice").value)}));
addRecord("routeForm","routes",()=>({
from:document.getElementById("routeFrom").value.trim(),
to:document.getElementById("routeTo").value.trim(),
info:document.getElementById("routeInfo").value.trim()}));
document.getElementById("bookingForm").addEventListener("submit",e=>{
e.preventDefault();
if(sessionStorage.getItem("role")!=="user"){showMessage("Please log in first.");return;}
const booking={name:document.getElementById("guestName").value.trim(),
email:document.getElementById("guestEmail").value.trim(),
destination:document.getElementById("bookDestination").value,
hotel:document.getElementById("bookHotel").value,
date:document.getElementById("checkin").value,
guests:document.getElementById("guests").value};
const list=data("bookings");list.push(booking);write("bookings",list);
document.getElementById("bookingConfirmation").innerHTML=
'<div class="notice"><strong>Booking request saved!</strong><br>Guest: '+
escapeHTML(booking.name)+'<br>Destination: '+escapeHTML(booking.destination)+
'<br>Accommodation: '+escapeHTML(booking.hotel)+'<br>Check-in: '+
escapeHTML(booking.date)+'</div>';
e.target.reset();renderAll();
});
init();
</script>
</body>
</html>
```

8. Sample Input

Operation	Field	Sample value
Registration	Name / Email / Password	Anita / anita@example.com / travel123
Admin login	Email / Password	admin@wanderlust.com / admin123
Add destination	Name / Description	Hampi / Historic temples and monuments
Add food	Food / Location	Local thali / Hampi
Add hotel	Name / Place / Price	Temple View Stay / Hampi / 1600
Add route	From / To / Info	Bengaluru / Hampi / Bus – approx. 7 hours
Booking	Destination / Hotel / Date / Guests	Ooty / Hill View Residency / 2026-12-15 / 2

9. Expected Output with Image

When index.html is opened in a browser, the home page displays the Wanderlust Travels title, navigation menu, a travel banner and popular destination cards. Users can open the registration and login pages, browse the available details, and submit a booking after logging in. The admin dashboard displays management forms and the booking list.

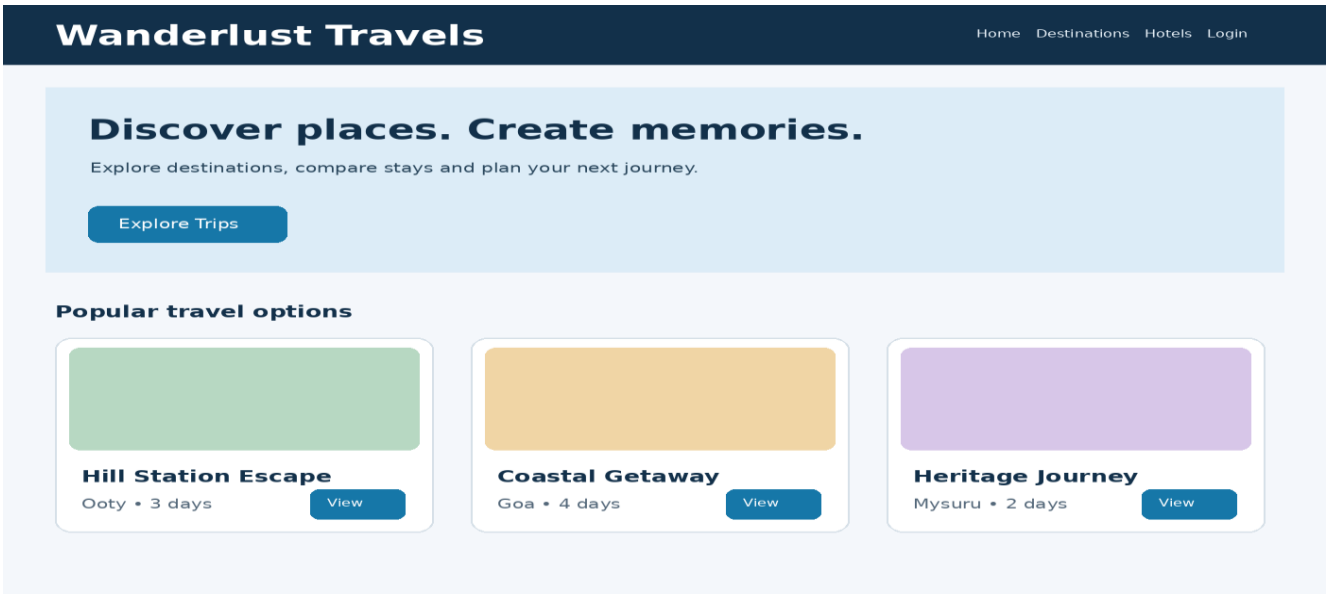


Figure 1: Illustrative expected home-page output of Wanderlust Travels.

10. Result

Thus, the Wanderlust Travels online travel agency website was designed and developed successfully using HTML, CSS and JavaScript. The prototype demonstrates user registration and login, destination/food/hotel/route viewing, accommodation booking, and admin management of travel information. The expected output was obtained in a web browser.